LVA-1 — Deflake CredentialsViewModelTest > select provider updates selectedProvider

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** feature/credentials/src/test/kotlin/lava/feature/credentials/CredentialsViewModelTest.kt **Severity:** P1

67th-cycle full-suite flaky test (fixed-awaitState-count vs Room Flow .first() off the StandardTestDispatcher). Fixed in the 68th cycle (Commit 1) via bounded await-until-selectedProvider loop; falsifiability-rehearsed. **Source:** self-discovered — .lava-ci-evidence/sixth-law-incidents/2026-05-20-flaky-credentialsviewmodeltest.json

LVA-2 — §6.X-debt darwin/arm64 emulator-acceleration sub-debt

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/sixth-law-incidents/2026-05-13-emulator-container-darwin-arm64-gap.json **Severity:** P1

Per-OS emulator acceleration (Containers c1871138 + 6aff7ea8): macOS gate runner is host-direct+HVF. PROVEN by C00 cold-start canary + full 37-class Challenge suite on Pixel_8/API35 (43 pass / 3 credential-skip / 0 fail). RESOLVED 2026-05-20. **Source:** self-discovered — .lava-ci-evidence/sixth-law-incidents/2026-05-13-emulator-container-darwin-arm64-gap.json

LVA-009 — HelixQA VM QA launch-dispatch exit 127 (vision works, action-dispatch command-not-found)

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** submodules/helixqa pin 639f7652 (launch verb builds launcher intent) + .lava-ci-evidence/helixqa/vm-qa-vision2-20260608T182251Z (vision drives app, exit-127 gone) **Severity:** P2 **Created-By:** AI

After the 3-layer claude vision bridge fix (helixqa aef46e5d), the HelixQA autonomous vision QA GENUINELY drives the Lava app on the Genymotion VM: it analyzes screenshots and decides actions (vision-screen-changed PASS; rationale 'Current screen is

the Android home launcher … launching the Lava client app'). Remaining blocker: the 'launch' action dispatch returns 'exit status 127' (command not found) — visionnav step 2 dispatch 'launch digital.vasic.lava.client.dev'. The derived launch action is 'shell monkey -p digital.vasic.lava.client.dev 1'; the ADB actor appears to run it without the 'adb -s ' prefix (treating 'shell' as a command) OR adb is not on the dispatch subprocess PATH. Fix in helixqa pkg actor/bridge ADB dispatch. Evidence: .lava-ci-evidence/helixqa/vm-qa-vision2-20260608T182251Z/ (4/4 reached vision+navigation, 1 ERROR + 3 FAILED on the launch-dispatch 127, NOT bridge errors anymore).

LVA-010 — lava-api-go router mounts /v1/{provider} group WITHOUT provider middleware (handlers panic→500)

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** commit ec85b753 lava-api-go/internal/router/router_provider_dispatch_test.go (ProviderMiddleware mounted; /v1 dispatch 200/404/501 not 500) **Severity:** P2 **Created-By:** AI

internal/router/router.go Build() registers the /v1/:provider route group without the provider-resolution middleware, so those handlers panic-recover to 500 in production instead of resolving a provider. Surfaced by the new router_test.go (W4) which asserts resolution (non-404) — the routes resolve but to a 500, not a working provider dispatch. Confirm whether production wires the middleware elsewhere (real server bootstrap) or this is a real gap. Evidence: router_test.go note + internal/router/router.go.

LVA-011 — TestEndpointsRepository.add Third-Law divergence (stores Rutracker; real impl no-ops it)

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** commit d9a4eaaa core/testing TestEndpointsRepository Rutracker no-op + equivalence test **Severity:** P3 **Created-By:** AI

core/testing TestEndpointsRepository.add() stores an Endpoint.Rutracker and can raise a duplicate-conflict for it, but the real EndpointsRepositoryImpl.add() early-returns for Rutracker (if (endpoint is Endpoint.Rutracker) return) and never stores it. A future test asserting 'adding Rutracker is a no-op' would pass against the fake while exercising different behavior

than production — a latent §Third-Law bluff-fake. Fix: add the Rutracker no-op branch + doc to the fake. Flagged by the W6 core/domain UseCase agent (2026-06-09).

LVA-012 — core/testing TestVisited/Favorites/Bookmarks repositories are TODO-throw stub bluff-fakes

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** commit d28ddd22 core/testing Visited/Favorites/Bookmarks fakes implemented + 13 equivalence tests **Severity:** P2 **Created-By:** AI

core/testing TestVisitedRepository, TestFavoritesRepository, TestBookmarksRepository have observe/contains/add/etc methods that are TODO('Not yet implemented') (throw). They are unusable for behavioral wiring — feature ViewModel tests (account, visited, rating) work around them with local in-memory fakes. Per the Third Law these shared fakes should be behaviorally-equivalent to the real repos so tests can use them directly. Flagged by the rating+visited VM-test agent (2026-06-09). Fix: implement the stubs as real in-memory fakes matching the real repo contracts + add equivalence tests.

LVA-014 — TestSuggestsRepository was a TODO-throw stub (LVA-012 bluff class)

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** core/testing TestSuggestsRepository implemented (in-memory, case-insensitive UPSERT id=lowercase().hashCode(), newest-first, clear) + TestSuggestsRepositoryTest (4 tests); Bluff-Audit: drop dedup filterNot → 'case-insensitive UPSERT MUST keep one row expected:<1> but was:<2>', reverted; core:testing 31/0, feature:menu 17/0 **Created-By:** AI

core/testing TestSuggestsRepository.observeSuggests/addSuggest threw TODO(); clear() no-op. Real SuggestsRepositoryImpl emits newest-first (timestamp DESC) + case-insensitive UPSERT (id=lowercase().hashCode(), REPLACE) + clear deletes all. Unusable stub = Third-Law bluff fake.

LVA-015 — TestSearchHistoryRepository positional-id + no-UPSERT + wrong ordering (Third-Law divergence + bluff test)

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** core/testing TestSearchHistoryRepository content-derived id (replicates Filter.id()) + UPSERT + newest-first; TestSearchHistoryRepositoryTest rewritten to real contract (6 tests); Bluff-Audit: revert to positional-append → 'upserts the same logical search' + 'dedups searches that differ only by sort' FAILED expected:<1> but was:<2>, reverted; core:testing 31/0, feature:menu 17/0, feature:search 9/0 **Created-By:** AI

core/testing TestSearchHistoryRepository.add used positional id (it.size) + always appended (no dedup) + insertion order, while real SearchHistoryRepositoryImpl uses content-derived Filter.id() + @Insert REPLACE UPSERT + ORDER BY timestamp DESC. The existing TestSearchHistoryRepositoryTest asserted the fake-shaped behavior (ids [0,2], oldest-first) — a bluff test.

LVA-016 — RatingViewModelTest RealObserveRatingRequestUseCase drops the engagement gate (ACTIVE bluff)

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** core/domain ObserveRatingRequestUseCaseImpl made public (Fifth Law); feature/rating RatingViewModelTest now wires the REAL use case over LVA-012/015-fixed shared fakes, seeds real engagement (3 bookmarks>2) in the 4 Show-path tests. Bluff-Audit: neutralize seedEngagement → 4 Show-path tests FAIL 'TurbineAssertionError: No value produced in 3s' (real engagement gate the old fake dropped), 2 Hide tests pass; reverted. GREEN feature:rating 6/0, core:domain 57/0 **Created-By:** AI

feature/rating RatingViewModelTest's local RealObserveRatingRequestUseCase re-implements ObserveRatingRequestUseCaseImpl but OMITS the 3rd condition (engagement: pinned>1 OR other>3 OR visited>5 OR bookmarks>2). The test 'Show rating request is rendered when conditions are met' seeds ZERO engagement + asserts Show, but the REAL use case would emit Hide for that user state. Second-Law/§6.J bluff: re-implements internal business logic + asserts an outcome production would not produce. Fix: wire the real ObserveRatingRequestUseCaseImpl (deep tree:

ObserveSearchHistory/Visited[->EnrichTopics]/Bookmarks use cases over shared in-memory fakes, now usable post LVA-012/015) + seed engagement in the Show test. Source: parallel fake-audit 2026-06-09.

LVA-017 — feature favorites/topic local fakes: add() lacks REPLACE dedup (LATENT)

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** feature/favorites InMemoryFavoritesRepository.add + feature/topic FakeFavoritesRepository.add now dedup-by-id (REPLACE) + 2 falsifiability tests. Bluff-Audit: revert to append → 'expected:<[7]> but was:<[7, 7]>', reverted; feature:favorites 6/0, feature:topic 4/0 **Created-By:** AI

feature/favorites InMemoryFavoritesRepository.add + feature/topic FakeFavoritesRepository.add append without dedup, while FavoritesRepositoryImpl/FavoriteTopicDao are @Insert REPLACE (id PK). LATENT: no test re-adds a duplicate id, so unexercised. Tighten add() to replace-by-id for consistency with LVA-011..015. Source: parallel fake-audit 2026-06-09.

LVA-018 — core/preferences dead getters getHistorySyncPeriod/getCredentialsSyncPeriod (zero call sites)

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** core/preferences PreferencesStorage[Impl] getHistorySyncPeriod/getCredentialsSyncPeriod removed (zero call sites, proven) + 2 test-fake overrides removed; core:preferences 22/0, core:auth:impl 14/0; values still surfaced via getSettings() **Created-By:** AI

PreferencesStorage.getHistorySyncPeriod()/getCredentialsSyncPeriod() (interface + Impl) have zero production call sites; sync-period values are surfaced via getSettings() instead. Orphaned per-field-getter leftover (no Favorites/Bookmarks counterpart). Remove decl + override. Source: parallel dead-code audit 2026-06-09 (codebase otherwise clean: 0 shipped TODO(), 0 if(false), 0 silent-no-op effect methods).

LVA-7 — §11.4.85 stress + chaos test scaffold

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** lava-api-go/internal/handlers/v1/search_thundering_herd_test.go — 512 concurrent identical-key reqs through REAL read-through cache; asserts all-200 + post-burst upstream loads=0 (convergence) + no goroutine-leak + -race clean. Bluff-Audit: delete cache.Set in search.go → 'convergence FAILED: post-burst loads=1 (want 0)', reverted. First runnable §11.4.85 chaos dim beyond phase-1; evidence JSON committed. **Severity:** P2

§11.4.85 (new universal anchor) mandates a stress + chaos test class. Lava has no chaos/stress suite today. Assess + scaffold when pin is bumped. **Source:** self-discovered — .lava-ci-evidence/constitution-review/2026-05-31-68th-cycle-review.md

LVA-020 — archiveorg array-valued creator/title/year/date fails the WHOLE search/browse/topic response

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** flexString type (string|number|array→joined) in internal/archiveorg/flexstring.go; structs+map sites in search/browse/topic.go converted; flexstring_test.go. Bluff-Audit: route '[' away from array case → TestSearchResponse_ArrayValuedFields + TestMetadataResponse + TestFlexString FAIL, reverted; go test GREEN, api-source.hash regenerated, sourcehash contract GREEN **Created-By:** AI

internal/archiveorg search.go/browse.go/topic.go decoded creator/title/year/date as string/*string; archive.org returns these as JSON arrays for multi-author/date items → json.Unmarshal fails the ENTIRE response (every result), not one row. Found by parallel Go bug-hunt.

LVA-021 — observability error_class collapses to literal "error" for all real typed errors (telemetry blindness)

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** underlyingTypeName fallback → fmt.Sprintf(%T, err); nonfatal_typed_errorclass_test.go (unnamed typed + *fs.PathError chain + context sentinels). Bluff-Audit: revert to "error" → TestClassOf_UnnamedTypedError + _StdlibTypedError FAIL 'collapsed to error', reverted; GREEN **Created-By:** AI

internal/observability/nonfatal.go underlyingTypeName returned "error" for any error without Name() or the 2 context sentinels — i.e. virtually every real *fs.PathError/*net.OpError/*url.Error/pgx error → §6.AC per-type triage defeated. Existing nonfatal_errorclass_test was a bluff (only tested Name()-implementers).

LVA-022 — gutenberg bestFormatName returns blank label for charset-suffixed Gutendex MIME keys

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** matchFormatByPrefix helper; bestFormatName+pickBestFormatURL prefix-match; utils_charset_format_test.go. Bluff-Audit: disable charset prefix → TestBestFormatName_CharsetSuffixed + TestPickBestFormatURL FAIL, reverted; existing TestBestFormatName_Table still GREEN (PDF-label scope-creep reverted) **Created-By:** AI

internal/gutenberg/utils.go bestFormatName + pickBestFormatURL exact-matched bare MIME keys (text/plain) but Gutendex always suffixes charset (text/plain; charset=utf-8) → blank Format label + skipped preferred ordering for text-only books.

LVA-023 — v1 login returns User.Id (profile data-uid) as AuthToken instead of User.Token (session cookie)

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** provider.go AuthToken: success.User.Token; provider_adapter_e2e_test.go TestAdapter_Login_ReturnsSessionCookieNotDataUID_E2E (real POST /login.php→/index.php→/profile.php flow). Bluff-Audit: revert to User.Id → test FAILS 'AuthToken = the data-uid 99999', reverted; GREEN **Created-By:** AI

internal/rutracker/provider.go ProviderAdapter.Login returned success.User.Id (numeric data-uid) as AuthToken; the real session cookie is User.Token. Login 'succeeds' but every authenticated v1 call (favorites/add-comment/download) sends a non-cookie value → 401. No e2e login test existed (why it shipped).

LVA-024 — v1 GetTopic fabricates a bogus TopicFile{Name:"Size"} from the torrent size string

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** provider.go drops the synthetic Files entry; provider_mapping_test.go TestFromTopicPage rewritten to assert len(Files)==0. Bluff-Audit: restore the Size-file line → TestFromTopicPage FAILS 'want 0 entries', reverted; GREEN **Created-By:** AI

internal/rutracker/provider.go fromTopicPage wedged the size string into a synthetic file entry → topic detail screen shows one nonsense 'Size' file instead of the real list. The existing TestFromTopicPage asserted the bogus file (bluff).

LVA-027 — Kinozal search sizeBytes hardcoded null — every result drops its size

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** KinozalSizeParser (binary mult, comma/dot, Latin+Cyrillic units) + KinozalSearchParser sizeBytes wired; KinozalSizeParserTest 5 tests incl end-to-end. Bluff-Audit: revert sizeBytes→null → 'search row carries sizeBytes end to end FAILED', reverted; :core:tracker:kinozal:test GREEN **Created-By:** AI

KinozalSearchParser parsed the size string into a local var then emitted sizeBytes=null, so every Kinozal row dropped size (size sort/filter + cross-tracker ranking blind). Found by parallel Kotlin tracker bug-hunt.

LVA-025 — v1 captcha login sends the answer under the wrong form-field name (captcha login can never succeed)

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items/LVA-025-026-evidence.md **Created-By:** AI

internal/handlers/v1 LoginOpts has only CaptchaCode (used as the answer) but rutracker needs CaptchaCode=dynamic-field-NAME (cap_code_) + CaptchaValue=answer. The adapter sets both to the answer (self-acknowledged TODO provider.go:221). Needs LoginOpts + OpenAPI model change (CaptchaValue field) — deferred. Found by parallel Go bug-hunt.

LVA-026 — v1 captcha response hardcodes image/png, discards upstream Content-Type

Status: Implemented (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-025-026-evidence.md **Created-By:** AI

internal/handlers/v1/captcha.go serves c.Data(200, image/png, ...) but rutracker.FetchCaptcha captures the real Content-Type, dropped by the adapter (no ContentType field on provider.CaptchaImage). Minor (most decoders sniff). Found by parallel Go bug-hunt.

LVA-028 — Nnmclub search publishDate dropped (date column present + parseable)

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items/LVA-028-evidence.md **Created-By:** AI

NnmclubSearchParser reads row.select('td') but never maps cells[4] (ISO yyyy-MM-dd date) into TorrentItem.publishDate → Nnmclub results have no date. Found by parallel Kotlin tracker bug-hunt. (Also UNCONFIRMED: nnmclub/kinozal parseSize Latin-only regex may miss Cyrillic units in real HTML — kinozal already handles both as of LVA-027.)

LVA-029 — isLocalHost() fc/fd false-positive misclassifies public hosts as IPv6 unique-local

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items/LVA-029-evidence.md **Created-By:** AI

core/models HostUtils.isLocalHost runs the fc00::/7 unique-local check (startsWith fc/fd + take(4) hex in 0xfc00..0xfdff) on ANY host string without requiring an IPv6 literal, so a DNS host like fcba.example.com / fdcdn.net is routed as LAN (http://host:8080) instead of https://host/forum/ → no green dot, every request fails. Fix: gate on contains(':') before the hex parse. Found by parallel core/network bug-hunt.

LVA-6 — §11.4.79 reconcile codegraph index policy (own-org submodules IN index)

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/../codegraph/lva6-groundtruth-20260609.md **Severity:** P2

§11.4.79 (new) requires own-org submodules IN the codegraph index; Lava currently EXCLUDES submodules/ per docs/CODEGRAPH.md + 63rd-cycle policy. Reconcile .codegraph config + docs when pin is bumped. **Source:** self-discovered — .lava-ci-evidence/constitution-review/2026-05-31-68th-cycle-review.md

LVA-031 — Genymotion VM+nav UUIDs in tracked evidence trip §6.R scanner

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items/../codegraph/lva6-groundtruth-20260609.md **Severity:** P2

scan-no-hardcoded-uuid.sh flags VM UUID + NavBackStackEntry UUIDs in 14 tracked genymotion evidence files → check-constitution exit 1. Redacted per e767b701 precedent.

LVA-032 — rutracker browse/favorites blind-cast Topic→Torrent type confusion

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items/LVA-032-evidence.md **Severity:** P1

fromCategoryPage/fromFavoritesDto called AsForumTopicDtoTorrent ignoring the union discriminator → Topic variants render as empty fake-torrent rows. Fixed via AsForumTopicDtoTorrentChecked.

LVA-033 — rutor topic page drops Добавлен publishDate

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items/LVA-033-evidence.md **Severity:** P2

RuTorTopicParser dropped the Добавлен date despite the fixture carrying it. Added dotted DD-MM-YYYY shape to RuTorDateParser.

LVA-034 — Room endpoint-list converter drops GoApi platform/ storage/key on round-trip

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items/LVA-034-evidence.md **Severity:** P1

host:port-only packing dropped the per-instance key → list-selected on-device Lava-API endpoint 401s (withKeyOverride dead). Fixed via #-sentinel encoded packing.

LVA-035 — PagingDataLoader pagination state-machine had zero tests

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-035-evidence.md **Severity:** P2

Added PagingDataLoaderTest (append-terminal off-by-one + refresh-error regression), production byte-identical.

LVA-030 — 6 recently-added submodules missing §6.R inheritance pointer (pre-existing §6.AD-debt)

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-030-evidence.md **Created-By:** AI

doc_processor/helixqa/llm_orchestrator/llm_provider/llms_verifier/ vision_engine CLAUDE.md lack the §6.R No-Hardcoding inheritance block; scripts/check-constitution.sh full-run exits 1 on them. Pre-existing from the 5-dep + helixqa adoption; orthogonal to the 60e2d66 constitution bump. Each needs a submodule commit + push + parent pin bump (helixqa is always-track-upstream per Q9). Surfaced by full check-constitution during the constitution-bump cycle.

LVA-037 — §6.R hostport scanner flags its own tests/ hermetic fixture (missing tests/ exemption)

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items/LVA-037-evidence.md **Severity:** P2

scan-no-hardcoded-hostport.sh regex had /test/ but not top-level tests/ → flagged tests/check-constitution/ test_no_hardcoded_hostport.sh → check-constitution exit 1 → pre-push blocked. Added ^tests/ exemption.

LVA-038 — brotli middleware emitted Content-Encoding br on bodyless 204/304 (statusAllowsBody dead code)

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items/LVA-038-evidence.md **Severity:** P2

Wave-3 parallel-fleet find; fixed in f3b2d269; falsifiability-proven; main-stream re-verified.

LVA-039 — ProviderLoginViewModel.onReloadCaptchaClick leaves stale serviceUnavailable banner

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items/LVA-039-evidence.md **Severity:** P2

Wave-3 parallel-fleet find; fixed in 9f895502; falsifiability-proven; main-stream re-verified.

LVA-040 — onboarding anonymous-mode choice never persisted to provider_configs

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items/LVA-040-evidence.md **Severity:** P1

Wave-3 parallel-fleet find; fixed in 8e0720ab; falsifiability-proven; main-stream re-verified.

LVA-041 — SyncPeriod.DAY background-sync flex interval 6.days should be 6.hours (WorkManager clamp)

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items/LVA-041-evidence.md **Severity:** P2

Wave-3 parallel-fleet find; fixed in 0cecbe4e; falsifiability-proven; main-stream re-verified.

LVA-042 — SyncOutbox FIFO Third-Law fake divergence + createdAt regression coverage

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-042-evidence.md **Severity:** P2

Wave-3 parallel-fleet find; fixed in a11aee32; falsifiability-proven; main-stream re-verified.

LVA-043 — §6.E reverse-gate phantom-capability honesty assertion

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-043-evidence.md **Severity:** P2

Wave-3 parallel-fleet find; fixed in c117caeb; falsifiability-proven; main-stream re-verified.

LVA-045 — §6.AI-debt CM-COVENANT-114-* §11.4.128-141 propagation gate + slash-command docs

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-045-evidence.md **Severity:** P2

Wave-3 parallel-fleet find; fixed in c37995a7; falsifiability-proven; main-stream re-verified.

LVA-046 — rutracker Login reported fake success on wrong credentials (blind AuthResponseDto union accessor ignored discriminator)

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items/LVA-046-evidence.md **Severity:** P2

AsAuthResponseDtoSuccess() blind-decodes a WrongCredits union into non-pointer UserDto with no error → Login returns Success:true AuthToken:'' for wrong creds → every authed call 401s. LVA-032 class. Fixed via AsAuthResponseDtoSuccessChecked.

LVA-047 — MagnetLinkValidator rejects case-insensitive urn:btih prefix (RFC 8141)

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items/LVA-047-evidence.md **Severity:** P2

case-sensitive startsWith rejected magnet:?xt=urn:BTIH:; fixed to regionMatches ignoreCase.

LVA-044 — Add HTTP_DOWNLOAD TrackerCapability so archiveorg/ gutenberg can surface their working HTTP file download

Status: Implemented (→ Fixed.md) **Type:** Feature **Evidence:** .lava-ci-evidence/workable-items/LVA-044-evidence.md **Severity:** P3

archiveorg/gutenberg wire a real HTTP-download impl that is deliberately not exposed (no capability) → user-unreachable. Not a bluff (§6.E honest), a functionality gap. Needs a new capability + feature interface.

LVA-051 — §11.4.128 always-on device-recorder skeleton + deterministic-path test (device capture OWED)

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-051-evidence.md **Severity:** P2

scripts/record-device-session.sh + hermetic path test landed; device-bound capture UNCONFIRMED without a live device.

LVA-048 — feature/search_input writes author NAME under AuthorIdKey (wrong nav key)

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items/LVA-048-evidence.md **Severity:** P2

SearchInputNavigation openSearchInput appends filter.author?.name under AuthorIdKey instead of AuthorNameKey. Flagged by wave-4 §6.Q agent (out of its scope). Needs fix + roundtrip test.

LVA-049 — navigation route values not URL-encoded (reserved chars in search query corrupt route)

Status: Fixed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-049-evidence.md **Severity:** P3

appendOptionalParams passes raw query; a query with &/? corrupts the route. Add Uri.encode at call sites + roundtrip test.

LVA-050 — A11yContentDescriptionTest teardown RuntimeException (Robolectric RoboMonitoringInstrumentation) reds :core:designsystem:test despite assertions passing

Status: Fixed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-050-evidence.md **Severity:** P3

All 4 test bodies pass (XML failures=0) but a teardown RuntimeException makes Gradle report FAILED. Quarantine/fix the teardown.

LVA-053 — PostConverters.removeLast() crashes topic screen on Android <API 35 (NoSuchMethodError)

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items/LVA-053-evidence.md **Severity:** P1

Kotlin removeLast() desugars to java.util.List.removeLast (JDK21 SequencedCollection) absent on Android <35 → a PostBr followed by Hr crashes the topic screen. Fixed to removeAt(lastIndex) + 28 converter tests.

LVA-056 — §6.R IPv4 scanner flags device-recorder tests/ hermetic fixture (missing tests/ exemption)

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items/LVA-056-evidence.md **Severity:** P2

scan-no-hardcoded-ipv4.sh regex lacked top-level tests/ → flagged tests/device-recording synthetic 127.0.0.1:6555 → check-constitution exit 1 (LVA-037 class). Added ^tests/.

LVA-052 — Wire HTTP_DOWNLOAD into the topic/download UI (consume HttpDownloadableTracker, write artifact to disk)

Status: Implemented (→ Fixed.md) **Type:** Feature **Evidence:** .lava-ci-evidence/workable-items/LVA-052-evidence.md **Severity:** P3

HTTP_DOWNLOAD UI wiring. A full impl exists on branch worktree-agent-ad695086c8735d60a (commit c8951eee) but it re-derived LVA-044 substrate → conflicts with master's landed LVA-044; needs a clean re-apply of the LVA-052 net-new files (HttpDownloadSource/DownloadHttpFileUseCase/

DownloadService.downloadHttpFile/SDK.downloadHttpFile) on top of master. Topic-screen Compose routing (provider id through nav) still OWED.

LVA-054 — Audit codebase for removeLast/removeFirst/getFirst/getLast SequencedCollection calls that crash on Android <API35

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-054-evidence.md **Severity:** P1

LVA-053 class: Kotlin stdlib these desugar to java.util.* methods absent below API35. Sweep all modules + add a lint/detekt guard.

LVA-055 — run-test-pg.sh integration list omits ./internal/storage (Postgres legs skipped by canonical harness)

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-055-evidence.md **Severity:** P3

scripts/run-test-pg.sh runs only ./internal/cache + ./tests/integration; add ./internal/storage so its Postgres legs run in the canonical harness.

LVA-057 — multi-search SSE silently drops unknown requested providers

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items/LVA-057-evidence.md **Severity:** P2

GetMultiSearch continued past an unknown ?providers= id with no provider_error event and no failed counter, while total_providers still counted it — the user's requested provider vanished with zero client signal. Now emits provider_error + counts failed.

LVA-058 — §6.R scanners flag binary workable_items.db SSoT (LVA-037/056 class)

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items/LVA-058-evidence.md **Severity:** P2

The ipv4/hostport/uuid source-literal scanners scanned the tracked binary SQLite SSoT docs/workable_items.db, which stores ticket text quoting IP/host/UUID literals, producing a Binary file matches violation. Added .db exemption to all three.

LVA-060 — favorite/visited Torrent reconstruction drops magnetLink and date

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items/LVA-060-evidence.md **Severity:** P1

FavoriteTopicEntity/VisitedTopicEntity toTopic() reconstructed Torrent vs BaseTopic without checking date/magnetLink, so a magnet-only favorited or visited torrent was read back as BaseTopic and became un-downloadable, and getTorrents() filtered it out entirely. Added both fields to the discriminator.

LVA-059 — sort ProviderRegistry.IDs() consumers for deterministic multi-search SSE provider ordering

Status: Fixed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-059-evidence.md **Severity:** P3

multi-search auto-discovery emits providers in non-deterministic map order (cosmetic).

LVA-061 — mobile/tls.go regenerate embed cert when persisted IP-SANs no longer cover device LAN IP

Status: Fixed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-061-evidence.md **Severity:** P3

self-signed cert reused across restarts without re-checking IP-SAN; LAN IP change → host-mismatch.

LVA-062 — audit gate-shaping scripts for uncovered clauses (each gate clause needs a branch-covering falsifiability sub-test)

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-062-evidence.md **Severity:** P3

wave-6 §6.N.2 found CM-WORKABLE-ITEMS-SYNC clause-3 was uncovered; sweep other gates.

LVA-063 — cover CredentialsViewModel + CredentialsManager vault lifecycle branches

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-063-evidence.md **Severity:** P3

Added 9 falsifiable tests for the credentials dialog SubmitDialog flows, §6.G verified filter, and the credentials_manager FirstTimeSetup/AddNew/Edit/DismissEdit lifecycle.

LVA-066 — validate new Jackett release against the Lava Torznab sidecar integration

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-066-evidence.md **Severity:** P2

Operator flagged a new Jackett version. Booted lscr.io/linuxserver/jackett v0.24.2040-ls426 via podman in the loopback topology: api_key bootstrap + Torznab /caps endpoint served valid XML at the exact path IPTorrentsJackettApi uses; JVM parser tests green; pinned the validated digest in .env.example.

LVA-064 — loadOrCreateTLS does not detect expired persisted cert (NotAfter passed)

Status: Fixed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-064-evidence.md **Severity:** P3

The TLS reuse gate checks IP-SAN coverage but not certificate expiry, so a persisted cert past its NotAfter is silently reused; add an expiry check to the reuse gate with a test.

LVA-065 — audit feature/search SearchViewModel for untested error/paging branches

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-065-evidence.md **Severity:** P3

SearchViewModel was not inspected in depth this loop; audit its error and paging-state branches and add falsifiable coverage where real gaps exist.

LVA-068 — proactively re-mint embed TLS leaf mid-process when it crosses the expiry margin

Status: Fixed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-068-evidence.md **Severity:** P3

tls.go only re-checks cert expiry at Start; a multi-month-running embed could serve a leaf that expires without a restart. Add a periodic re-mint check.

LVA-069 — cover SearchResultViewModel SSE raw-JSON parsing + onRetryClick re-dispatch

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-069-evidence.md **Severity:** P3

handleSseEvent raw-JSON parsing (provider_start/results/provider_done/provider_error) and onRetryClick Error-state re-dispatch are untested production branches.

LVA-073 — §6.AC telemetry instrumentation for the accepted LVA-008 nav-teardown crash

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-073-evidence.md **Severity:** P1

Per operator accept-with-telemetry decision, NavTeardownCrashReporter (chained uncaught handler) tags the known upstream androidx nav-teardown ISE with attributable §6.AC Crashlytics context before the process dies, so it surfaces as a known/triageable defect; 6 falsifiable JVM tests; wired into LavaApplication.

LVA-072 — §6.AC telemetry on mid-process embed TLS cert rotation (LVA-068 swap is silent)

Status: Fixed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-072-evidence.md **Severity:** P3

LVA-068 rotates the embed leaf mid-process but the swap is silent; emit a RecordWarning on rotation with feature/operation/old+new NotAfter/IP-SANs context (no secrets per §6.H). A wave-10 attempt was discarded because its falsifiability rehearsal was inconclusive and the SourceHash contract failed; redo with a test that fails when the actual rotation-path RecordWarning call is removed.

LVA-074 — Genymotion runner must wake+stay-on the VM screen before connectedDebugAndroidTest

Status: Fixed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-074-evidence.md **Severity:** P2

A sleeping VM screen idles the render pipeline causing SurfaceFlinger commit timeouts and spurious No-compose-hierarchies Challenge failures; the runner now wakes and keeps the screen on, proven by C00/C01/C07/C08 going green after the wake.

LVA-070 — thread source providerId through favorite/visited write+read path to complete HTTP_DOWNLOAD routing

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-070-evidence.md **Severity:** P3

LVA-067 laid the providerId Room column; complete it by threading providerId through ToggleFavoriteUseCase/ AddLocalFavoriteUseCase/GetTopicUseCase + the Topic/ TopicPage models (write) and TopicModel + favorites/visited side-effects to openTopic(id,providerId) (read), mirroring search_result.

LVA-071 — inject SseClient + base-URL into SearchResultViewModel.observeSseSearch for hermetic MockWebServer testing

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-071-evidence.md **Severity:** P3

observeSseSearch internally constructs the SseClient with a hardcoded https base; inject it so the full SSE error to Error to retry path is hermetically testable against a MockWebServer.

LVA-067 — persist per-row providerId on favorite/visited rows so their topics can route to HTTP_DOWNLOAD

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items/LVA-067-evidence.md **Severity:** P3

LVA-052 scoped favorites/visited topic downloads to the active-tracker default because the favorite/visited Room rows store no provider id; add a providerId column + migration so an archiveorg/gutenberg favorite routes to HTTP_DOWNLOAD.

LVA-4 — LVA workable-items ticket system (SQLite DB + Go CLI + md/HTML/PDF/DOCX export)

Status: Implemented (→ Fixed.md) **Type:** Feature
Evidence: .lava-ci-evidence/workable-items/LVA-4-evidence.md
Severity: P1

HelixConstitution §11.4.93/95/106 materialization. Go CLI (modernc.org/sqlite, no CGO) with init/add/update/close/reopen/gen/verify/import/export. Operator directive §6.L 68th invocation, key prefix LVA. Superseded by migration to the canonical constitution binary (docs/tickets/MIGRATION-TO-CANONICAL.md). **Source:** operator-report — docs/tickets/DESIGN.md

LVA-076 — apiengine processJavaRes buildCshared task dependency + Gradle heap

Status: Completed (→ Fixed.md) **Type:** Bug **Evidence:** docs/CONTINUATION.md **Severity:** medium

Gradle validation BUILD FAILED on clean .cxx: process JavaRes consumed buildCshared output without a declared edge; D8 OOM on 2g heap. Declared dependency plus bumped heap to 4g.

LVA-077 — release cold-start canary script for the R8 release artifact

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** docs/CONTINUATION.md **Severity:** medium

App has no connectedReleaseAndroidTest because testBuildType is debug; added scripts run-release-canary.sh to validate the exact R8 release APK cold-start on the Genymotion VM per clause 6.Z.

LVA-078 — restore lava-api-go container image build broken containerignore

Status: Completed (→ Fixed.md) **Type:** Bug **Evidence:** docs/CONTINUATION.md **Severity:** medium

Root containerignore blanket-excluded submodules but go.mod replaces parent submodules X; in-container go build failed on missing replacement dirs. Excluded only nested git and build, kept source.

LVA-092 — Video #11 — Negative finding: NO crash/ANR/white-icon observed in this recording (recorded for completeness)

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/video-analysis/2026-06-25-lava-issues-video.md **Severity:** P3 **Created-By:** AI

QA video 0001-0155: app did not crash/ANR; brand logo renders red. LVA-008 search-back crash + Sync-toggle crash + §6.AB white-icon were NOT triggered on screen here (remain open under their own tickets). Informational record. Source: .lava-ci-evidence/video-analysis/2026-06-25-lava-issues-video.md #11.

LVA-080 — Distribute 1076/1.3.12 (+ api-app 22): build_and_release -> §6.Z C00 gate on thinker -> §6.AA two-stage Firebase distribute

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/distribute-changelog/firebase-app-distribution-api-app/0.2.11-22-test-evidence.md **Severity:** P1 **Created-By:** AI **Assigned-To:** AI

1076 built but never distributed (last-distributed=1075). Operator directive: rebuild, gate, distribute all 4 variants. Infra confirmed UP this session: thinker+nezha reachable, T7 writable, JDK17/Gradle8.9/podman5.8.2 present.

LVA-036 — llm_orchestrator github↔gitlab pre-existing mirror fork (non-FF) blocks §6.W convergence

Status: Fixed (→ Fixed.md) **Type:** Task **Evidence:** docs/issues/fixed/BUGFIXES.md **Severity:** P2

github/master and gitlab/master diverged at d2a2151 with unique non-doc go.mod content each; LVA-030 commit landed gitlab+working-tree but github refused non-FF. Needs a content-merge decision (operator-gated, NO force-push per §6.T.3).

LVA-083 — Video #1 — Search returns ZERO results then 'Something went wrong' Error (primary function unusable)

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/lva-1082-device-gate/Challenge58SearchReturnsResultsTest/real-device-verification.json + firebase-distribute.sh release 2eadee3bf6vq0 (1.3.15-1082, distributed 2026-08-12) **Severity:** P0 **Created-By:** AI **Assigned-To:** AI

QA video 2026-06-25 frames 0060-0140: every search fails (blank ~25s then Error/Retry; 'prince' stays blank). KNOWN-class (anonymous/provider-mismatch). CODE-FIX landed in 1076 (SearchInputViewModel observeAll filtered+sorted + loading/empty state) but 1076 NOT yet distributed/device-verified. Pending §6.Z gate. Source: .lava-ci-evidence/video-analysis/2026-06-25-lava-issues-video.md #1.

LVA-084 — Video #2 — Onboarded provider (YTS) is NOT the provider set used by Search; unconfigured providers active as filters

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/lva-1082-device-gate/Challenge59SearchUsesOnboardedProvidersTest/real-device-verification.json + firebase-distribute.sh release 2eadee3bf6vq0 (1.3.15-1082, distributed 2026-08-12) **Severity:** P0 **Created-By:** AI **Assigned-To:** AI

QA video frames 0030 vs 0040/0060: onboarded only YTS but search used RuTracker/RuTor/IA/Gutenberg etc. KNOWN (§6.L 57th/59th). CODE-FIX in 1076 (chips from ProviderConfigRepository.observeAll() searchEnabled&&isEnabled). Pending §6.Z device verification. Source: .lava-ci-evidence/video-analysis/2026-06-25-lava-issues-video.md #2.

LVA-013 — Missing 6.Z device evidence for client 1080 and api-app 24

Status: Obsolete (→ Fixed.md) **Type:** Task **Evidence:** Superseded: 1080/api-app-24 were never distributed and never will be. 1082/0.2.12-25 now have real §6.Z device-gate + release-canary evidence and are fully distributed (both stages) as of 2026-08-12. **Obsolete-Details:** Since: 2026-08-12; Reason: superseded-by-later-mandate; Superseding-item: 1.3.15-1082 + 0.2.12-25 distribute cycle; Triple-check evidence: .lava-ci-evidence/distribute-changelog/firebase-app-distribution/1.3.15-1082-test-evidence.md + .lava-ci-evidence/distribute-changelog/firebase-app-distribution-api-app/0.2.12-25-test-evidence.md

Task P0 Android: the 1080 client and 24 api-app cycles were distributed without per-AVD containerized emulator evidence. Need to execute the covering Challenge matrix, generate real-device-verification rows, and backfill the evidence files.

LVA-085 — Video #4 — Provider id labels shown raw/lowercased ('torrentdownloads','archiveorg','kinozal','yts') in results filter chips

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** feature/search_result/src/test/kotlin/lava/search/result/SearchResultViewModelStreamingTest.kt::streaming_search_seeds_providerDisplayNames_on_first_Streaming_state_before_any_ProviderStart_event — real ViewModel+SDK+registry test, falsifiability rehearsal performed (mutation reverted, real observed AssertionError). Root cause: SearchResultScreen.ProviderFilterChipBar rendered from providerDisplayNames map populated only by async ProviderStart events, arriving after the first composed frame. Fix: eager synchronous seed via sdk.listAvailableTrackers() in the same reduce that opens Streaming state. Commit 80c9380d. **Severity:** P1 **Created-By:** AI **Assigned-To:** AI

QA video frames 0060/0125/0130: results chips render internal provider key not displayName. KNOWN-class (§6.L 60th displayLabel). CODE-FIX in 1076 (friendly chip names). Pending §6.Z verification. Source: .lava-ci-evidence/video-analysis/2026-06-25-lava-issues-video.md #4.

LVA-086 — Video #5 — No empty-state and no loading indicator on search results (perceived hang)

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** feature/search_result/src/test/kotlin/lava/search/result/SearchResultViewModelLoadingEmptyStateTest.kt — 3 real Challenge tests, falsifiability rehearsal performed. Root cause: render predicate for the loading/empty state existed inline in SearchResultScreen but was never extracted/tested, so a prior fix (e7b6a652) could not be verified. Fix: extracted SearchPageState.streamingFilteredItems + showsStreamingLoadingIndicator as named, tested properties. Commit ffc0fb25. **Severity:** P1 **Created-By:** AI **Assigned-To:** AI

QA video frames 0060-0110: pure blank ~25s, no spinner/skeleton/no-results; 'prince' stays blank with no error. NEW. CODE-FIX in 1076 (loading/empty state branches). Pending §6.Z verification. Source: .lava-ci-evidence/video-analysis/2026-06-25-lava-issues-video.md #5.

LVA-093 — Cold-start race: search can hit bundled/direct client before dynamic provider repopulation completes

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** core/domain/src/main/kotlin/lava/domain/usecase/StartupProvidersGate.kt (new) + RepopulateProvidersOnStartupUseCase.kt + SearchResultViewModel.observeStreamMultiSearch — 15 real tests across StartupProvidersGateTest/RepopulateProvidersOnStartupUseCaseTest/SearchResultViewModelStreamingTest, 3 falsifiability rehearsals performed. Fix: bounded StateFlow readiness gate (5s timeout fallback) awaited before the tracker client is resolved. Commit 9d85c5cf. **Severity:** P2 **Created-By:** AI

app/src/main/kotlin/digital/vasic/lava/client/LavaApplication.kt:95-105 launches RepopulateProvidersOnStartupUseCase.repopulateProviders() fire-and-forget on Dispatchers.Default inside

Application.onCreate() -- it is not awaited. app/src/main/kotlin/digital/vasic/lava/client/MainActivity.kt:105-135 gates the splash screen ONLY on local prefs (theme/showOnboarding) loading and has no dependency on repopulateProviders() completion. Consequently the splash can dismiss and the user can reach the search screen while the network round-trip inside core/domain/src/main/kotlin/lava/domain/usecase/RepopulateProvidersOnStartupUseCase.kt:80-112 is still in flight. If the user searches during that window, any provider id the catalogue vends that overlaps a BUNDLED compiled-in provider id (rutracker/rutor/nnmclub/kinozal/archiveorg/gutenberg -- see core/tracker/client/src/main/kotlin/lava/tracker/client/di/TrackerClientModule.kt:297-303) resolves to the direct-to-site bundled client for that one search instead of the user's configured API endpoint. Once the fetch completes, all later searches in that session are correctly dynamic -- this is a real, narrow-window, self-healing defect, not a permanent break. No existing test covers this exact race: the closest test, RepopulateProvidersOnStartupUseCaseTest, exercises the use case in isolation and does not measure or force the Application.onCreate-to-first-interactive-search timing window. CONFIRMED by static source-reading this session (2026-08-10/11), part of the LVA-083/084 root-cause investigation that also found the OnboardingBypassRule-skips-populateFrom test bug in Challenge58/59/60/61/62/71. Real-world frequency and user-visible impact are UNCONFIRMED without device instrumentation -- no build or test was executed for this specific finding, only source reading. Closure needs either (a) a device-level Challenge Test that measures or deliberately forces this race and asserts correct provider resolution once repopulation completes, or (b) a product decision to gate the splash screen / first search on repopulation completion (with a timeout + fallback) instead of racing it.

LVA-094 — Cold-start provider repopulation failure is silent and never retried for the rest of the process lifetime

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** core/domain/src/main/kotlin/lava/domain/usecase/RepopulateProvidersOnStartupUseCase.kt — try/finally markReady() guarantee + fetchWithRetry() single retry + recordFetchFailure() §6.AC non-fatal telemetry per failed attempt. Real RepopulateProvidersOnStartupUseCaseTest coverage, falsifiability rehearsal performed (reverted to single-attempt/no-telemetry, both retry-rescue and telemetry tests failed for real). Commit 9d85c5cf. **Severity:** P2 **Created-By:** AI

RepopulateProvidersOnStartupUseCase's single cold-start provider-catalogue fetch attempt fails silently on failure: RepopulateProvidersOnStartupUseCase.kt:110 returns false with no user-visible notice (unlike onboarding's own fetch-failure path, which surfaces a PROVIDER_CATALOG_FALLBACK_NOTICE), no retry, and no other production code path re-invokes populateFrom() for the rest of that process's lifetime. Confirmed: the only two production call sites for populateFrom are OnboardingViewModel.kt:950-998 (onboarding flow) and this startup use case (app cold start) -- there is no periodic worker and no connectivity-change listener that re-triggers it. A transient failure at exact boot time (network not yet associated, VPN/Wi-Fi still connecting, momentarily unreachable LAN lava-api-go/api-app endpoint) means a user who configured a non-bundled API endpoint during onboarding silently falls back to bundled/direct-to-site tracker clients for the ENTIRE app session, with zero indication anything degraded, until the process restarts (a fresh attempt) or the user re-runs onboarding. Fix direction, documented as the finding only and not yet implemented: either add a retry affordance / silent background retry with backoff, or at minimum record a non-fatal telemetry event per this project's own section 6.AC Comprehensive Non-Fatal Telemetry Mandate so the failure is visible in Crashlytics instead of invisible. CONFIRMED by static source-reading this session (2026-08-10/11), same investigation as the sibling cold-start race finding LVA-093. Real-world frequency and user-visible impact are UNCONFIRMED without device instrumentation -- no build or test was executed for this specific finding, only source reading.

LVA-081 — Fetch/pull/merge latest from all submodules + build-verify (operator directive 2026-06-26)

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/bluff-hunt/2026-08-12-lva-081-submodule-bump-evidence.md **Severity:** P2 **Created-By:** AI **Assigned-To:** AI

Operator directive: fetch+pull+merge latest codebase from all submodules. Frozen-by-default overridden for this cycle. Must build-verify after each bump; revert+report any submodule whose bump breaks the build.

LVA-3 — Adopt HelixConstitution §11.4.79–106 (pin 53 commits behind)

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/bluff-hunt/2026-08-12-lva-3-constitution-adoption-review.md **Severity:** P1

Pin 208e2c8 is 53 commits behind origin/main 883ccc1. Highest-impact new clauses: §11.4.93/95/106 (workable-items SQLite DB tracked in git + md to DB sync engine), §11.4.79 (own-org submodules in CodeGraph), §11.4.85 (stress/chaos), §11.4.98, §11.4.102. Pin-bump is operator-gated; decision owed on §6.AD.3 Path B vs SQLite DB. **Source:** self-discovered — .lava-ci-evidence/constitution-review/2026-05-31-68th-cycle-review.md

LVA-087 — Video #6 — Welcome claims '4 providers available' but picker lists ~12

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/bluff-hunt/2026-08-12-lva-087-device-verification-evidence.md **Severity:** P2 **Created-By:** AI **Assigned-To:** AI

QA video frames 0007/0010/0015 vs 0020-0030. NEW. CODE-FIX in 1076 (#6 count bound to real descriptor list). Pending §6.Z verification. Source: .lava-ci-evidence/video-analysis/2026-06-25-lava-issues-video.md #6.

LVA-090 — Video #9 — Onboarding 'Select all' silently enables auth-requiring (Captcha/Form Login) providers

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/genymotion/c66-2026-08-12-lva090-reverify/verdict.txt **Severity:** P3 **Created-By:** AI **Assigned-To:** AI

QA video frames 0020-0025. NEW UX, contributes to #1. CODE-FIX in 1076 (#9 select-all handling): OnboardingViewModel.onToggleAllProviders() + requiresNoCredentials() correctly gate select-all to no-auth-only providers, cited to Issue #9 in-source. UNIT-LEVEL RE-VERIFIED 2026-08-12 (this cycle): OnboardingViewModelVideoFixesTest 'select all enables only no-credential providers' PASS, fresh execution (timestamp 2026-08-12T21:07, not cached). Existing DEVICE-LEVEL evidence at .lava-ci-evidence/genymotion/c66-

redesign-{RED,GREEN}-20260626/ (proper RED-then-GREEN falsifiability pair, Challenge66SelectAllDoesNotEnableAuthProvidersTest) is 48 days / ~7 versions stale per SS6.AK -- device gate was occupied by a parallel LVA-087 verification agent this cycle, could not re-run. Source: .lava-ci-evidence/video-analysis/2026-06-25-lava-issues-video.md #9.

LVA-079 — Video #3 — search-input chips vs results-filter chips disagree + results chip set non-deterministic run-to-run

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/bluff-hunt/2026-08-12-lva-079-verification-summary.md **Severity:** P1 **Created-By:** AI **Assigned-To:** AI

QA video 2026-06-25 (frames 0040 vs 0060 vs 0125): input chip bar and results filter chips show different provider sets, and the results chip set CHANGES between two identical queries. 1076 fixed the INPUT chips (observeAll filtered+sorted) but the input-vs-RESULTS divergence + run-to-run instability is distinct and still open. Source: .lava-ci-evidence/video-analysis/2026-06-25-lava-issues-video.md issue #3.

LVA-095 — on-device API keyless mDNS endpoint does not read/persist local api-app key

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items-closures/2026-08-20-lva-095-lva-082-closure.md **Severity:** P1 **Created-By:** AI

OnboardingViewModelDynamicProvidersTest > 'selecting a keyless on-device API reads and persists the local api key for search auth' FAILS: AssertionError - the local api-app key MUST be read + persisted onto the keyless mDNS-selected endpoint (was GoApi(host=localhost.localdomain, port=42873, platform=null, storage=null, key=null)). Discovered 2026-08-12 while running the combined test suite after merging 4 parallel LVA-085/086/087/093/094 fixes - confirmed genuinely pre-existing and unrelated to any of those 4 changes (git log shows the test file's last touch was commit b3cb6de2, predating this session; none of the 4 merged branches touch onboarding/api-key/mDNS

code per git diff --name-only). Not yet root-caused - source: real gradle test run, feature/onboarding/build/test-results/testDebugUnitTest/.

LVA-082 — Crashlytics read NOT available via Firebase CLI — only symbols/mappingfile upload; no issues:list; bq absent

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items-closures/2026-08-20-lva-095-lva-082-closure.md **Severity:** P3 **Created-By:** AI

Operator asked to use Firebase CLI to pull Crashlytics. Verified Firebase CLI 14.17.0 exposes only crashlytics upload + mappingfile:* (NO issue/non-fatal read). bq CLI absent. Crashlytics dashboard read requires console or a BigQuery export not configured. Fallback: in-repo §6.AC telemetry + known crash tickets; operator to paste console items for full triage. §11.4.6 honest record.

LVA-088 — Video #7 — 'Choose your API' shows 'lava.app:7777' preset + mislabeled 'On this network'

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items-closures/2026-08-20-lva-088-089-091-closure.md **Severity:** P2 **Created-By:** AI **Assigned-To:** AI

QA video frames 0012/0015. 1076 investigation: NOT a §6.R hardcoding violation (config-driven preset). 'On this network' label for a cloud/remote preset still worth confirming. Pending §6.Z verification. Source: .lava-ci-evidence/video-analysis/2026-06-25-lava-issues-video.md #7.

LVA-089 — Video #8 — mDNS-discovered API shows raw IP 192.168.0.107:8443 with no friendly name

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items-closures/2026-08-20-lva-088-089-091-closure.md **Severity:** P2 **Created-By:** AI **Assigned-To:** AI

QA video frames 0012/0015. NEW UX. CODE-FIX in 1076 (discovered-API friendly name). Pending §6.Z verification. Source: .lava-ci-evidence/video-analysis/2026-06-25-lava-issues-video.md #8.

LVA-091 — Video #10 — App-ID co-mingling (debug .dev + release both labeled 'Lava') — UNCONFIRMED in video

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** .lava-ci-evidence/workable-items-closures/2026-08-20-lva-088-089-091-closure.md **Severity:** P3 **Created-By:** AI

QA video frames 0001/0005: single Lava icon launched; co-mingling NOT visually confirmed. Needs on-device package check (applicationIdSuffix .dev + launcher label). OPEN/UNCONFIRMED. Source: .lava-ci-evidence/video-analysis/2026-06-25-lava-issues-video.md #10.

LVA-019 — Coverage ledger partial/gap overlap and missing per-release ledgers

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .lava-ci-evidence/workable-items-closures/2026-08-20-lva-019-closure.md

Task P1 process: coverage ledger has partial/gap overlap and lacks per-release ledger snapshots. Normalize the registry and add release-attestation ledgers.

LVA-096 — Pipeline 002 — report.json evidence_summary never populated, making the anti-bluff PASS gate a no-op

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_run_report_evidence_summary.sh **Severity:** P1 **Created-By:** AI

WHAT: init_run_report in scripts/pipeline/lib/run-report.sh seeded report.json evidence_summary to all zeros and nothing ever updated it. phase-02-test.sh tallied the identical counters locally and printed them to the console only. finalize_run_report gates outcome PASS on evidence_summary.rejected_by_anti_bluff == 0, so a counter permanently stuck at 0 turned the anti-bluff half of the data-model.md Validation rule into a silent no-op: a run whose

Evidence Records had been REJECTED could still finalize to PASS. DEMONSTRATED (verbatim RED): "FAIL: outcome is 'PASS', expected FAIL - a run containing a REJECTED Evidence Record reported success." FIX: new recompute_evidence_summary() in scripts/pipeline/lib/run-report.sh derives every count from the physical Evidence Records on disk instead of a counter a phase must remember to increment; the orchestrator calls it before finalize on every exit path including interrupt. Bluff-Audit: mutating the aggregator REJECTED* match to a never-matching literal reproduced the original bluff exactly; reverting restored all-green with zero regressions across the three pre-existing pipeline suites. Found during T038. Severity high. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-097 — Pipeline 002 — orchestrator self-diagnosis missed its own dirtied tree (porcelain collapses untracked dirs)

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_pipeline_orchestrator.sh **Severity:** P3 **Created-By:** AI

WHAT: git status --porcelain collapses a fully-untracked directory to a single entry, so the orchestrator first-cut diagnosis for the "pipeline dirtied its own tree" footgun matched nothing at all when the offending paths were inside one untracked directory. FIX: use --untracked-files=all. Re-verified against a fixture repo with the T003 gitignore rule removed, which then correctly exits 2 AND prints the DIAGNOSIS block naming both offending paths. Found during T038. Severity low. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-098 — Pipeline 002 — companion doc under docs/scripts for a scripts/pipeline script BREAKS CM-SCRIPT-DOCS-SYNC

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** docs/scripts/pipeline/phase-06-docs.sh.md **Severity:** P2 **Created-By:** AI

WHAT: a companion doc placed at docs/scripts/.sh.md for a script living under scripts/pipeline/ breaks the CM-SCRIPT-DOCS-SYNC gate rather than satisfying it. The gate matches find scripts -maxdepth 1 -name '*.sh*' *against find docs/scripts -maxdepth 1*

-name '.sh.md', so such a doc is an ORPHAN with no depth-1 script counterpart. DEMONSTRATED mechanically: exit 1 with "Docs WITHOUT matching scripts/: docs/scripts/phase-05a-changelog-entry.sh.md". FIX: pipeline phase-script companions go in docs/scripts/pipeline/, following the existing docs/scripts/{hooks,lava-api-go,autonomous-qa}/ precedent, which sits outside the maxdepth-1 scan. Gate confirmed clean afterwards (68 scripts to 68 docs, 1:1). Found during T042/T044/T045. Severity medium. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-099 — Pipeline 002 — guard-regression test could start a REAL Gradle build on the developer checkout

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_pipeline_orchestrator.sh **Severity:** P2 **Created-By:** AI

WHAT: while rehearsing a mutation of the orchestrator "--skip precondition" refusal, the guard stopped refusing, so the run fell through to the DEFAULT --until live_verify and began a REAL Gradle build against the REAL repository, which had to be killed at a 2-minute timeout. A guard-regression test that can start a real build on the actual checkout when the guard regresses is itself a hazard. FIX: every Group A guard invocation in tests/pipeline/test_pipeline_orchestrator.sh is now aimed at a throwaway fixture repo and executed from inside it, so the same regression fails fast and harmlessly. Found during T038. Severity medium. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-100 — Pipeline 002 — constitutional-gate-sweep wrapper was dead code — no _dispatch line in phase-02-test.sh

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** scripts/pipeline/phase-02-test.sh **Severity:** P1 **Created-By:** AI

WHAT: scripts/pipeline/phase-02-test-constitutional-gate-sweep.sh was fully implemented and functional but had NO _dispatch line in scripts/pipeline/phase-02-test.sh, which carried six _dispatch calls and none for this category. The constitutional-gate-sweep category was therefore silently absent from every run - not skipped-with-a-reason, just missing - while the run summary read as complete.

The tolerate-absence path documented in that file header did NOT cover it: that path protects a category whose _dispatch line exists but whose script file does not; a category with no _dispatch line at all is invisible to it. FIX: added GATE_SWEEP_WRAPPER plus its _dispatch call to the first (non-Gradle) parallel group, PROVEN to fire by an isolated run through the file own PHASE02_GATE_SWEEP_WRAPPER override against a marker-writing stub, confirming the documented argument order reaches the wrapper. Found during T028. Severity high. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-101 — Pipeline 002 — jq // fallback erased boolean false, turning all_passed:false into "unknown"

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** scripts/pipeline/phase-02-test-constitutional-gate-sweep.sh **Severity:** P2 **Created-By:** AI

WHAT: jq -r '.all_passed // "unknown"' falls back not only on null/missing but also on boolean false, so a real Containers attestation saying "all_passed": false was recorded in every Evidence Record as "unknown" - erasing the did-not-all-pass signal on exactly the runs where it matters. FIX: explicit has(...) and != null form that still distinguishes genuinely-absent from present-and-false; unit-verified mappings false to false, true to true, null to unknown, {} to unknown. The sibling // uses on the count fields are unaffected (jq treats 0 as truthy - verified by execution, not assumed). Found during T028. Severity medium. This shape recurs as S-D in the wave-4 wrapper audit and is Check B of the standing static guard. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-102 — Pipeline 002 — phase-02-test.sh reported PASS on ZERO Evidence Records (vacuous pass)

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_phase_02_aggregation.sh **Severity:** P1 **Created-By:** AI

WHAT: the phase-02-test.sh PASS policy had three conditions (no wrapper exited non-zero, no record said FAIL, no record was REJECTED) and every one is satisfied VACUOUSLY by a run in which the dispatched wrappers all exit 0 without writing anything. Such a run genuinely reported "PASSED - all 1 dispatched

wrapper(s) exited 0, 0 Evidence Records scanned, 0 FAIL, 0 REJECTED" and the run report carried it forward as a passing phase. That file own header already states the governing principle ("an empty test phase proves nothing") but enforced it only for the zero-WRAPPERS case, not the zero-EVIDENCE case that actually reaches the aggregator. DEMONSTRATED (verbatim RED): "FAIL: zero Evidence Records -> exit 0. A test phase that scanned no evidence at all reported success." FIX: a fourth PASS condition requiring at least one Evidence Record scanned, plus an explanatory failure message. New hermetic suite tests/pipeline/test_phase_02_aggregation.sh (3 cases) drives the real aggregator through the file own PHASE02_*_WRAPPER override hooks with stub wrappers. Case 1 (honest wrapper, one real record, PASS) and Case 3 (zero wrappers, pre-existing refusal) stayed green throughout, so the fix is not a blanket fail-everything. Found during T028 follow-up. Severity high. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-103 — Pipeline 002 — hermetic wrapper would have minted a PASS record from a reproduce-a-known-bug harness

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** scripts/pipeline/phase-02-test-hermetic.sh **Severity:** P2 **Created-By:** AI

WHAT: tests/firebase/run_all.sh globs test_*.sh, so a repro_ file is outside its reach. But scripts/pipeline/phase-02-test-hermetic.sh enumerates ALL *.sh at depth 2 via find tests -mindepth 2 -maxdepth 2 -name* '.sh', so it WOULD have run tests/firebase/repro_mode_both_channel_gap.sh as a hermetic suite. Because that harness exits 0 when it SUCCESSFULLY REPRODUCES a defect, the pipeline would have minted a PASS Evidence Record whose real meaning is "a known bug is still broken". That is strictly worse than the redundant-aggregator case the wrapper header already guards against: that one over-counts real coverage, this one would manufacture coverage out of a known failure. FIX: exclusion rule 3 in the hermetic wrapper, -not -name 'repro_*.sh' (scripts/pipeline/phase-02-test-hermetic.sh line 242), with the reasoning documented alongside the two existing rules. Proven by the enumeration count going 1 to 0 for repro files while the wrapper still enumerates 69 real suites; its own hermetic test passes and tests/firebase/run_all.sh remains 8/8. Found while adding the reproduction harness for the firebase-distribute combined-mode finding. Severity medium. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-104 — Pipeline 002 — the anti-bluff validator was ITSELF bluffable — every rule passed on an empty record

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_anti_bluff_missing_evidence_fields.sh **Severity:** P0 **Created-By:** AI

WHAT: the deepest defect in this feature. Every one of the four rules in scripts/pipeline/lib/anti-bluff-validate.sh was satisfied by a record carrying NO evidence at all. An empty assertion_summary contains none of the rule-1 bluff phrases. An empty raw_output_ref resolves to ${record_dir}/ - the record OWN DIRECTORY - and both [[-e dir]] and [[-s dir]] are TRUE for a directory (its inode has non-zero size), so rules 2 and 3 passed on nothing. An empty category is not "real-device-challenge", so rule 4 was skipped. On top of that, jq -r '.x // empty' exits 0 for an ABSENT field, so the rule-0 guard never fired despite its header claiming to reject records missing required fields. A sibling of the same shape: rule 4 is selected by exact string match, so a category typo "real-device-challenges" skipped the mandatory falsifiability-marker check entirely. REACHABLE THROUGH THE PRODUCTION WRITER, not only via hand-edited JSON: any wrapper passing its raw DIRECTORY instead of a captured-output file produced validated / exit=0 on empty evidence. WHY IT MATTERS: this component is the one FR-004 relies on to catch a test lying about itself, so a bluffable anti-bluff validator makes every downstream "validated" stamp in the pipeline unfalsifiable. FIX: rule 0c (every schema-required field present AND non-empty, checked BEFORE any rule an empty value would satisfy), rule 0d (category must be one of the 8 enum values), and rule 2 tightened from -e to -f with a distinct message for the directory case. New suite tests/pipeline/test_anti_bluff_missing_evidence_fields.sh (8 checks including 3 over-correction guards). Independently re-verified: a directory-valued raw_output_ref now yields "REJECTED: raw_output_ref 'raw' is a directory, not a captured-output file", an absent field yields "REJECTED: required field(s) missing or empty: assertion_summary (a record that asserts nothing and points at nothing is not evidence)", and the ORIGINAL validator suite still passes. Found by the systematic vacuous-PASS audit; task T012. Severity critical. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-105 — Pipeline 002 — phase-01-build.sh recorded and counted Build Artifacts that were NOT on disk

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_phase_01_artifact_verification.sh **Severity:** P1 **Created-By:** AI

WHAT: _phase01_build_android_report_one ran cp -f "$src" "$dst" with its exit status UNCHECKED, then unconditionally printed ARTIFACT : and returned 0. scripts/pipeline/phase-01-build.sh believed that line, recorded the path into build_artifacts[] and incremented its counter, also without checking the _record_build_artifact result. The PASS condition (android_rc==0 && go_rc==0 && artifacts>=5) was satisfied entirely by the ABSENCE of the check that would have caught it. DEMONSTRATED with a real run against a read-only destination: "cp: cannot create regular file ...: Permission denied" immediately followed by "ARTIFACT app-debug: ...", then "5 of 5 expected Build Artifacts recorded", "all 5 Build Artifacts produced and recorded", PHASE EXIT=0 - while the recorded path DOES NOT EXIST. This matters downstream: scripts/pipeline/phase-02-test.sh resolves the release-canary APK out of exactly that array. SECOND, DIFFERENT-CLASS DEFECT in the same run: _record_build_artifact read the version from app/build.gradle.kts and the commit from git rev-parse HEAD in $REPO_ROOT, while the sub-builders built from the [repo-path] override, so an artifact built from a fixture at 9.9.9 was recorded as 1.3.17/1086 - recorded metadata that does not describe the recorded artifact. FIX: cp status checked plus post-copy -f/-s verification; each declared path verified real and non-empty before recording; only successful records counted with unverified ones folded into the verdict; metadata read from the same path the build used. New suite tests/pipeline/test_phase_01_artifact_verification.sh (11 checks, including an all-real build still passing and a zero-APK build still failing). Task T020. Severity high. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-106 — Pipeline 002 — phase-04 live-verify PASSED on an EMPTY HTTP response body

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_phase_04_live_verify_response_body.sh **Severity:** P1 **Created-By:** AI

WHAT: the header of scripts/pipeline/phase-04-live-verify-api.sh states it exists to prove the service genuinely answers REAL application-level HTTP requests with REAL, meaningful response bodies - not merely that its process/container lifecycle is Up (constitution 6.B). The per-request verdict was computed from the curl exit code and a 2xx match ONLY; the body was read, truncated to 300 chars, interpolated into assertion_summary, and never asserted on. DOUBLY VACUOUS: anti-bluff rule 3 (raw_output_ref must be non-empty) was satisfied by the scaffolding lines THIS SCRIPT ITSELF writes into the raw file, present whether or not the server said anything. DEMONSTRATED against a real TLS server returning 200 with a zero-byte body on every path: 3 requests attempted, all 3 records anti_bluff_status=validated, phase result PASS, PHASE EXIT=0, with the summary ending "...returned HTTP 200 with response body: " - nothing after the colon. FIX: a whitespace-stripped empty body now fails the request with a specific reason. Over-correction guarded by pinning the three endpoint real bodies, read from lava-api-go internal/observability/health.go and internal/router/router.go rather than invented. New suite tests/pipeline/test_phase_04_live_verify_response_body.sh (9 checks). Task T036. Severity high. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-107 — Pipeline 002 — wave4 G-1: go test exit code captured and never compared

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_phase_02_go_wrapper.sh **Severity:** P1 **Created-By:** AI

WHAT: scripts/pipeline/phase-02-test-go.sh captured the real go test exit code into a variable and never compared it. A package whose tests all PASS but which fails AS A PACKAGE - a TestMain teardown after m.Run(), which is exactly how the lava-api-go integration suites are written - reported "go test exited 1 ... PASS: 1 FAIL: 0 ... PASSED" with wrapper exit 0. FIX: the wrapper now writes a synthetic FAIL record quoting the real package-level output. Part of the T021-T027 post-implementation wrapper audit: 11 confirmed swallowed-failure defects, all fixed RED-then-GREEN. No real suite was run - each wrapper was driven through its own documented [repo-path] [phase-dir] seam against synthetic fixtures (stub gradlew, stub run-chaos-stress.sh, stub run-release-canary.sh, a PATH-level stub go), with the Go fixtures captured verbatim from this host real go1.26.2 toolchain and every Go fix re-verified against it afterwards. Each new suite carries positive cases so a blanket fail-everything fix would not

pass. Severity high. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-108 — Pipeline 002 — wave4 G-2: zero parseable go test events reported PASS with zero Evidence Records

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_phase_02_go_wrapper.sh **Severity:** P2 **Created-By:** AI

WHAT: scripts/pipeline/phase-02-test-go.sh reported PASS with zero Evidence Records when zero per-test events were parseable. The header promised exit 3 for this case; the implemented guard only tested whether the JSONL file was empty, so a stray build tag excluding every _test.go sailed through as "[no test files]", exit 0. FIX: the wrapper now genuinely exits 3. Part of the T021-T027 post-implementation wrapper audit: 11 confirmed swallowed-failure defects, all fixed RED-then-GREEN. No real suite was run - each wrapper was driven through its own documented [repo-path] [phase-dir] seam against synthetic fixtures (stub gradlew, stub run-chaos-stress.sh, stub run-release-canary.sh, a PATH-level stub go), with the Go fixtures captured verbatim from this host real go1.26.2 toolchain and every Go fix re-verified against it afterwards. Each new suite carries positive cases so a blanket fail-everything fix would not pass. Severity medium. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-109 — Pipeline 002 — wave4 K-1: GRADLE_EXIT_CODE captured and never compared

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_phase_02_kotlin_wrapper.sh **Severity:** P1 **Created-By:** AI

WHAT: scripts/pipeline/phase-02-test-kotlin.sh captured GRADLE_EXIT_CODE and never compared it. Because the wrapper correctly passes --continue, Gradle exits non-zero for failures that produce NO JUnit XML at all - a test-source compile error being the everyday case. An "Unresolved reference" compile failure reported "PASS: 1 / FAIL: 0", exit 0. FIX: the captured Gradle exit code is now compared and drives the verdict. Part of the T021-T027 post-implementation wrapper audit: 11 confirmed swallowed-failure defects, all fixed RED-then-GREEN. No real suite was run - each wrapper was driven through its own documented [repo-path] [phase-dir] seam against synthetic

fixtures (stub gradlew, stub run-chaos-stress.sh, stub run-release-canary.sh, a PATH-level stub go), with the Go fixtures captured verbatim from this host real go1.26.2 toolchain and every Go fix re-verified against it afterwards. Each new suite carries positive cases so a blanket fail-everything fix would not pass. Severity high. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-110 — Pipeline 002 — wave4 K-2: unparseable JUnit XML reports silently dropped

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_phase_02_kotlin_wrapper.sh **Severity:** P2 **Created-By:** AI

WHAT: scripts/pipeline/phase-02-test-kotlin.sh silently dropped unparseable report files. A truncated JUnit XML whose readable content contained a real produced "WARN: failed to parse ... unclosed token" and then "PASS: 1 / FAIL: 0", exit 0. A JUnit XML is truncated precisely when the JVM writing it died, so this is the highest-signal case being discarded. FIX: one FAIL record per unreadable report, quoting the parse error and the file real first bytes. Part of the T021-T027 post-implementation wrapper audit: 11 confirmed swallowed-failure defects, all fixed RED-then-GREEN. No real suite was run - each wrapper was driven through its own documented [repo-path] [phase-dir] seam against synthetic fixtures (stub gradlew, stub run-chaos-stress.sh, stub run-release-canary.sh, a PATH-level stub go), with the Go fixtures captured verbatim from this host real go1.26.2 toolchain and every Go fix re-verified against it afterwards. Each new suite carries positive cases so a blanket fail-everything fix would not pass. Severity medium. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-111 — Pipeline 002 — wave4 K-3: reports present but zero parsed exited 0 with zero records

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_phase_02_kotlin_wrapper.sh **Severity:** P2 **Created-By:** AI

WHAT: scripts/pipeline/phase-02-test-kotlin.sh exited 0 with zero Evidence Records when report files were present but zero elements parsed out of them - a scanned-nothing run reported as a clean run. FIX: zero parsed testcases is now a failure rather

than a vacuous pass. Part of the T021-T027 post-implementation wrapper audit: 11 confirmed swallowed-failure defects, all fixed RED-then-GREEN. No real suite was run - each wrapper was driven through its own documented [repo-path] [phase-dir] seam against synthetic fixtures (stub gradlew, stub run-chaos-stress.sh, stub run-release-canary.sh, a PATH-level stub go), with the Go fixtures captured verbatim from this host real go1.26.2 toolchain and every Go fix re-verified against it afterwards. Each new suite carries positive cases so a blanket fail-everything fix would not pass. Severity medium. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-112 — Pipeline 002 — wave4 H-1: empty suite enumeration reported exit 0 with zero records

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_phase_02_hermetic_wrapper.sh **Severity:** P2 **Created-By:** AI

WHAT: when suite enumeration in scripts/pipeline/phase-02-test-hermetic.sh matched nothing runnable, the wrapper printed "0 suite(s) to run ... WRAPPER EXIT CODE = 0" and wrote zero Evidence Records. (The real repo enumerates 76 candidates / 11 excluded / 65 run, so the guard cannot misfire in production.) FIX: an empty enumeration is now a failure. Part of the T021-T027 post-implementation wrapper audit: 11 confirmed swallowed-failure defects, all fixed RED-then-GREEN. No real suite was run - each wrapper was driven through its own documented [repo-path] [phase-dir] seam against synthetic fixtures (stub gradlew, stub run-chaos-stress.sh, stub run-release-canary.sh, a PATH-level stub go), with the Go fixtures captured verbatim from this host real go1.26.2 toolchain and every Go fix re-verified against it afterwards. Each new suite carries positive cases so a blanket fail-everything fix would not pass. Severity medium. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-113 — Pipeline 002 — wave4 H-2: skipped suites left no Evidence Record at all

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_phase_02_hermetic_wrapper.sh **Severity:** P2 **Created-By:** AI

WHAT: the scripts/pipeline/phase-02-test-hermetic.sh header claimed skips are "never silently omitted", but the only reporting was a console line that never reaches the run report, so a skipped suite was invisible at rest. FIX: one SKIPPED Evidence Record per excluded suite, quoting the wrapper own real exclusion reason. Part of the T021-T027 post-implementation wrapper audit: 11 confirmed swallowed-failure defects, all fixed RED-then-GREEN. No real suite was run - each wrapper was driven through its own documented [repo-path] [phase-dir] seam against synthetic fixtures (stub gradlew, stub run-chaos-stress.sh, stub run-release-canary.sh, a PATH-level stub go), with the Go fixtures captured verbatim from this host real go1.26.2 toolchain and every Go fix re-verified against it afterwards. Each new suite carries positive cases so a blanket fail-everything fix would not pass. Severity medium. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-114 — Pipeline 002 — wave4 S-A/S-B: empty or unreadable dimensions[] reported PASSED

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_phase_02_stress_chaos_wrapper.sh **Severity:** P2 **Created-By:** AI

WHAT: scripts/pipeline/phase-02-test-stress-chaos.sh reported PASSED with exit 0 on an empty or unreadable dimensions[] array - even when the truncated JSON own readable content said "verdict":"FAIL". FIX: an empty or unreadable dimensions array is now a failure rather than a pass. Part of the T021-T027 post-implementation wrapper audit: 11 confirmed swallowed-failure defects, all fixed RED-then-GREEN. No real suite was run - each wrapper was driven through its own documented [repo-path] [phase-dir] seam against synthetic fixtures (stub gradlew, stub run-chaos-stress.sh, stub run-release-canary.sh, a PATH-level stub go), with the Go fixtures captured verbatim from this host real go1.26.2 toolchain and every Go fix re-verified against it afterwards. Each new suite carries positive cases so a blanket fail-everything fix would not pass. Severity medium. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-115 — Pipeline 002 — wave4 S-C: RUN_RC captured and never compared

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_phase_02_stress_chaos_wrapper.sh **Severity:** P2 **Created-By:** AI

WHAT: scripts/pipeline/phase-02-test-stress-chaos.sh captured RUN_RC and never compared it: all-PASS dimensions plus a harness exit 1 reported PASSED, although the wrapper own header states that exit 0 additionally asserts no goroutine or file-descriptor leak. FIX: the harness exit code is now compared and drives the verdict. Part of the T021-T027 post-implementation wrapper audit: 11 confirmed swallowed-failure defects, all fixed RED-then-GREEN. No real suite was run - each wrapper was driven through its own documented [repo-path] [phase-dir] seam against synthetic fixtures (stub gradlew, stub run-chaos-stress.sh, stub run-release-canary.sh, a PATH-level stub go), with the Go fixtures captured verbatim from this host real go1.26.2 toolchain and every Go fix re-verified against it afterwards. Each new suite carries positive cases so a blanket fail-everything fix would not pass. Severity medium. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-116 — Pipeline 002 — wave4 S-D: .ran read as a string, so a MISSING key matched the same branch

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_phase_02_stress_chaos_wrapper.sh **Severity:** P2 **Created-By:** AI

WHAT: scripts/pipeline/phase-02-test-stress-chaos.sh read .ran as a string and compared it to "true", which also matches a MISSING key (null), so an unrecognized dimension shape was downgraded to a non-blocking operator-gated skip: a FAILing dimension printed "S1 sustained: null" and the wrapper reported PASSED. This is precisely the .all_passed // "unknown" defect class found in the gate-sweep wrapper, wearing a different disguise, and is Check B of the standing static guard. FIX: the flag is now read and compared in a form that distinguishes absent from present-and-false. Part of the T021-T027 post-implementation wrapper audit: 11 confirmed swallowed-failure defects, all fixed RED-then-GREEN. No real suite was run - each wrapper was driven through its own documented [repo-path] [phase-dir] seam against synthetic fixtures (stub gradlew, stub run-chaos-stress.sh, stub run-release-canary.sh, a PATH-level stub go), with the Go fixtures

captured verbatim from this host real go1.26.2 toolchain and every Go fix re-verified against it afterwards. Each new suite carries positive cases so a blanket fail-everything fix would not pass. Severity medium. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-117 — Pipeline 002 — wave4 R-A: undocumented exit codes laundered into SKIPPED, silently disabling the cold-start gate

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_phase_02_release_canary_wrapper.sh **Severity:** P1 **Created-By:** AI

WHAT: scripts/run-release-canary.sh documents exits 0/1/2 and runs under set -euo pipefail, so a missing tool yields 127 and a kill yields 137. scripts/pipeline/phase-02-test-release-canary.sh reported BOTH as "Genuinely did not execute: ... (real host/config precondition gap, not a feature defect)" with result SKIPPED and wrapper exit 0 - asserting a cause it cannot know. Since SKIPPED does not block the phase, a dead canary harness silently disabled the constitution 6.Z cold-start gate. FIX: only the documented exit 2 maps to SKIPPED; any other non-zero is a FAIL. Part of the T021-T027 post-implementation wrapper audit: 11 confirmed swallowed-failure defects, all fixed RED-then-GREEN. No real suite was run - each wrapper was driven through its own documented [repo-path] [phase-dir] seam against synthetic fixtures (stub gradlew, stub run-chaos-stress.sh, stub run-release-canary.sh, a PATH-level stub go), with the Go fixtures captured verbatim from this host real go1.26.2 toolchain and every Go fix re-verified against it afterwards. Each new suite carries positive cases so a blanket fail-everything fix would not pass. Severity high. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-118 — Pipeline 002 — advance-all-submodules.sh would have auto-advanced the constitution governance submodule

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_advance_all_submodules.sh **Severity:** P0 **Created-By:** AI

WHAT: scripts/advance-all-submodules.sh defaulted to "every submodule" (its own LAVA_ADVANCE_SUBMODULES documentation says so) and contained ZERO mention of the constitution submodule. Root CLAUDE.md states: "The ./constitution/ submodule itself remains pinned + advanced only per CONST-049 7-step pipeline." An unattended closure phase would therefore have auto-advanced the project own governance submodule - the document set that defines what this pipeline is permitted to do at all. Found by VERIFYING a claim (F-5) made in the T048/T049 constitutional-amendment drafts rather than relaying it, and found BEFORE the script had ever been run against a real submodule. DEMONSTRATED (RED): against a fixture whose submodule is named constitution with a newer upstream commit available, the pre-fix run recorded outcome=ADVANCED, parent_pin_updated=true, and the pin genuinely moved. FIX: a GOVERNANCE_DENY default-deny list checked FIRST in the loop - before fetch, before comparing commits (scripts/advance-all-submodules.sh line 193 onward). Deliberately NOT overridable via LAVA_ADVANCE_SUBMODULES, because an allow-list is a convenience for narrowing a run and a convenience switch must not be able to turn off a governance boundary. Matches on the path final component so both constitution and submodules/constitution are covered, verified against this repo real git submodule status output. A denied submodule still gets a Submodule Advance Record with a new REFUSED_GOVERNANCE_DENY outcome added to the contract schema, because a governance refusal that leaves no evidence is invisible at rest; old_commit and new_commit are both the current pin, since no candidate commit was ever looked up. SECOND BUG SURFACED BY THE FIX: the script carried its own internal outcome whitelist separate from the schema, which rejected the new value with "internal error - invalid outcome", so both had to move. Severity critical. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-119 — Pipeline 002 — close the vacuous-pass defect CLASS mechanically with a standing static guard

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** tests/pipeline/test_no_vacuous_pass_patterns.sh **Severity:** P2 **Created-By:** AI

WHAT: nineteen-plus defects found during feature 002 were overwhelmingly the same family - the "vacuous pass", a gate whose success conditions are all satisfied by the ABSENCE of the

thing it is supposed to check. Fixing the individual bugs is not the same as closing the class, so the recurring shapes were turned into a standing static guard over scripts/pipeline/** and the orchestrator: tests/pipeline/test_no_vacuous_pass_patterns.sh. CHECK A: an exit code captured into a variable and then never compared (seen in production in phase-02-test-go.sh, phase-02-test-kotlin.sh GRADLE_EXIT_CODE, phase-02-test-stress-chaos.sh RUN_RC). CHECK B: a jq // fallback applied to a field whose meaningful value can be falsy (seen in phase-02-test-constitutional-gate-sweep.sh and phase-02-test-stress-chaos.sh). CHECK C: -e or -s standing in for -f on a value that must be a regular file (seen in anti-bluff-validate.sh). ESCAPE HATCH: an inline "# vacuous-pass-ok: " comment on the same line, which requires the reasoning to be written down rather than allowing a silent exemption. Six legitimate cases were annotated this way rather than silenced, most importantly the phase-03 HEALTH_RC, where NOT trusting the return code is itself the load-bearing anti-bluff decision (constitution 6.B: the status binary exits 0 while reporting "Healthy: false", proven for real during T035). FALSIFIABILITY PROVEN: all three checks were rehearsed by re-introducing a real historical defect and confirming detection - the jq // bug (Check B), a bare -s on a raw file ref (Check C), and a stripped vacuous-pass-ok annotation on HEALTH_RC (Check A); each reverted byte-identically and the scanner returned to green. SCANNER OWN DEFECT FOUND AND FIXED: on its first real run, Check B flagged the explanatory COMMENT in the gate-sweep wrapper that quotes the very bug it fixed; a scanner that reports the documentation of a fixed bug as the bug is a false-positive generator, so it now skips comment lines. HONEST LIMITS, recorded rather than glossed: this is a heuristic lint, not a proof. It cannot catch every vacuous pass - the phase-01 "recorded an artifact that is not on disk" defect is invisible to it - and it is a cheap net for three shapes that have each already reached production here, not a substitute for the behavioural suites alongside it. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-139 — Pipeline 002 — the orchestrator trusted a phase to record its own failure, so report.json printed outcome PASS for a run that halted at test

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_phase_failure_always_recorded.sh **Severity:** P0

WHAT: The phase registry in scripts/pipeline-build-test-distribute.sh marks each phase with a self-appends-result flag and the orchestrator TRUSTED that flag. A phase script that exits non-zero BEFORE reaching its own append_phase_result call appends nothing; the in-flight marker was correctly cleared, because the script RETURNED and so the run was not interrupted; and finalize_run_report's all-PASS rule was then satisfied vacuously by the phases that did report. EVIDENCE: outcome PASS and "halted at: test" printed in the SAME summary box for a run that failed at the test phase. The multi-script variant is worse: live_verify's first script appends a PASS, the second dies before appending, and phases[] comes out FULL-LENGTH and all-PASS, indistinguishable from a complete success. RELATIONSHIP TO LVA-130, stated so this is not read as a duplicate: LVA-130 is the INTERRUPTED case, which the in-flight marker closes. This defect sits one level above it and the marker cannot cover it, because the marker's whole semantic is "a script is executing right now" and a script that returns is by definition not interrupted. The gap was never interruption; it was trusting a script to have recorded its own failure. FIX, verified by direct inspection of the working tree: a new phase_nonpass_count function at scripts/pipeline/lib/run-report.sh line 687 reports how many non-PASS entries a named phase actually recorded, and the orchestrator consults it at lines 557 and 588 and appends a FAIL itself when the count is zero. Deliberately NOT done there: no unconditional append, which would duplicate a phase that did report its own FAIL, and no default folded into the query, which would make "the report could not be read" and "the phase reported no failure" the same answer. An unreadable count is treated as zero on purpose, which appends, so the check fails closed. Regression coverage: tests/pipeline/test_phase_failure_always_recorded.sh.

LVA-140 — Pipeline 002 — six of seven unusable in-flight marker shapes read as not-interrupted, so an interrupted run finalized PASS

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_run_report_library_robustness.sh **Severity:** P1

WHAT: append_interrupted_phase_if_any in scripts/pipeline/lib/run-report.sh treated the in-flight marker's CONTENTS as the interruption signal when only its EXISTENCE is. Any marker the function could not interpret took a return-zero path that every caller reads as "this run was not interrupted", and the run then finalized PASS on a truncated all-PASS phases[]. MEASURED: six of the seven unusable marker shapes exercised took that path, namely an empty marker, a whitespace-only marker, an unreadable marker, a marker naming a phase the schema enum

rejects, a marker naming two phases, and a directory in the marker's place. That is the vacuous-pass shape the entire in-flight-marker block exists to close, reached through the block's own error handling. FIX, verified by direct inspection of the working tree: the marker's existence is now the signal and its contents only NAME the phase. The function returns 2 when a marker exists but neither it nor the caller-supplied fallback names a known phase, and it leaves the marker in place so the interruption is not lost; its documented contract now states that a non-zero return is never "nothing happened" and that callers MUST compare it. The fallback can only NAME an interruption the marker's existence already proves, never manufacture one, so with no marker present a fallback changes nothing. Separately, mark_phase_in_flight now writes through a temporary file, reads it back, and moves it into place atomically, so a failed or partial write no longer leaves an EMPTY marker behind: the marker either says which phase, or it is not there at all. Regression coverage: tests/pipeline/test_run_report_library_robustness.sh.

LVA-141 — Pipeline 002 — the orchestrator close path discarded the interrupted-phase library's failure signal, making could-not-record indistinguishable from completed

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** scripts/pipeline-build-test-distribute.sh **Severity:** P1

WHAT: the _close_report function in scripts/pipeline-build-test-distribute.sh invoked append_interrupted_phase_if_any with a trailing or-true, which discarded the library's return code. That made "a phase was in flight and its interruption could NOT be recorded" indistinguishable from "this run completed" at the point where the distinction matters most, and it defeated the library's return-code contract from the caller side. It is the same failed-measurement-reads-as-clean shape the library function itself was fixed for, one caller away. EVIDENCE: demonstrated with a fixture in which the library signalled failure and the orchestrator threw the signal away. FIX, verified by direct inspection of the working tree at scripts/pipeline-build-test-distribute.sh lines 411 to 419: the return code is captured into a local variable and COMPARED; a non-zero return sets an untrustworthy-report flag and prints an explicit operator-visible warning that a phase was IN FLIGHT, that its interruption could not be recorded, that the run did not finish, and that report.json must not be read as a pass. The successful case still prints which phase was interrupted and that it was recorded as FAIL.

LVA-142 — Pipeline 002 — recompute_evidence_summary aborted on the first uninterpretable Evidence Record under set -e, leaving the anti-bluff counter at zero

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_run_report_evidence_summary.sh **Severity:** P1

WHAT: recompute_evidence_summary in scripts/pipeline/lib/run-report.sh assigned each record field with a bare command substitution. A bare assignment carries the command's exit status, and this library's own header instructs callers to run under set -e with -u and pipefail, so the first malformed Evidence Record aborted the function mid-scan, evidence_summary was never written back, and it stayed at the all-zeros value init_run_report had left. A zero rejected-by-anti-bluff counter is exactly the condition finalize_run_report reads to allow PASS, so a run whose Evidence Records had been REJECTED could still finalize to PASS. SECOND HALF OF THE DEFECT, and the more damaging half: the function's own doc-comment stated that a record whose result cannot be interpreted "counts as FAILED, never silently ignored". That documented rule was FALSE. It held only for callers who happened to invoke the function on the left-hand side of an or-list, where set -e is suppressed. A documented protection that does not hold is worse than an absent one, because a reader stops looking. FIX, verified by direct inspection of the working tree: every field substitution now carries an explicit or-true, annotated in place as load-bearing rather than decorative, so the scan completes over every record and the documented counts-as-FAILED rule actually holds. Regression coverage: tests/pipeline/test_run_report_evidence_summary.sh.

LVA-143 — Pipeline 002 — the evidence-summary record selector counted a wrapper raw-output companion as a real Evidence Record

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** scripts/pipeline/lib/run-report.sh **Severity:** P2

WHAT: recompute_evidence_summary in scripts/pipeline/lib/run-report.sh selected Evidence Records with a per-phase-directory depth-two find for json files. Its doc-comment justified that depth by asserting that a wrapper's raw-output companions "always live one level deeper, under the category's raw directory, and are not

json". BOTH HALVES OF THAT CLAIM WERE FALSE. phase-02-test-constitutional-gate-sweep.sh writes its per-gate companions AS json files, where the depth filter happened to save the count; and nothing stopped a wrapper writing a companion into a raw directory placed directly under the phase directory, which IS at depth two and was therefore counted as a real Evidence Record, inflating both the total and the failed counters with a file that is not a record at all. IMPACT: an inflated failed counter is a false alarm rather than a false pass, so this is the milder direction, but the counters feed the run's outcome rule and a summary that miscounts in either direction is not evidence. It also means the outcome rule and the console SUMMARY could disagree without either being obviously wrong. FIX, verified by direct inspection of the working tree: the selector now carries an explicit exclusion of any path under a raw directory, so a raw-output companion is excluded because of WHERE it is rather than because of an assumption about its file extension, and the false doc claim is replaced by a dated correction in the same comment block that states plainly which half of the old claim was wrong.

LVA-144 — Pipeline 002 — the vacuous-pass static guard missed five capture forms in CHECK A and three grep forms in CHECK F

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_vacuous_pass_guard_discrimination.sh **Severity:** P2

WHAT: tests/pipeline/test_no_vacuous_pass_patterns.sh is the static guard that flags vacuous-pass shapes across the pipeline scripts, and two of its checks were narrower than their own stated purpose. CHECK A flags an exit code captured into a variable and never compared; its matcher recognised only the bare unquoted capture with an echo-shaped consumer, so five real forms escaped it: a QUOTED capture, a non-echo log helper standing in for echo as the consumer, a PIPESTATUS index other than zero, and the declare-with-integer-flag and local-with-readonly-flag declarator prefixes, with readonly and export in the same family. CHECK F flags a quiet grep on the right-hand side of a pipe under pipefail; it missed three forms: the long spelling of the quiet flag, a pipe at end-of-line where the consumer sits on the next physical line, and a digit-bearing intervening flag such as a max-count flag placed before the quiet flag. NET EFFECT: the guard reported a clean sweep over code that contained the very patterns it names, so the guard itself was the vacuous pass, and the earlier echo-only narrowing of CHECK A was an incomplete fix rather than a complete one. FIX, verified by direct inspection of the working tree: CHECK A's extractor and its candidate matcher both accept the optional declarator prefix, the quoted form and any

PIPESTATUS index; CHECK F joins continued lines before matching and accepts the long quiet spelling and digit-bearing intervening flags. TWO ESCAPES ARE LEFT OPEN DELIBERATELY, with measured reasons recorded in the guard's own known-uncovered section rather than silently: CHECK C on a variable not named with a reference-style suffix, for which zero true positives are available today, and CHECK B across a multi-line jq filter, where correct line joining needs quote tracking and CHECK F's cheap join has no safe analogue. One reported false positive was withdrawn before it was acted on, because the flagged line does consume its status by handing it to an embedded parser; the check was taught to tell a standalone argument from a variable embedded in prose rather than annotating a file outside the guard's surface. Regression coverage: tests/pipeline/test_vacuous_pass_guard_discrimination.sh.

LVA-145 — Pipeline 002 — CASE 3 of the interrupted-run test examined zero items and stayed green against a live cross-run marker leak

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_interrupted_run_never_passes.sh **Severity:** P1

WHAT: CASE 3 of tests/pipeline/test_interrupted_run_never_passes.sh asserts that the in-flight marker is per-run and never leaks between runs. It leaned on the premise that an earlier run in the same file had been left interrupted, and that premise was not true: CASE 1's call to append_interrupted_phase_if_any CONSUMED that run's marker, because removing the marker is how the operation is made idempotent, and CASE 2 cleared its own. By the time CASE 3 ran there was no marker anywhere on disk, so the assertion had nothing to discriminate against and examined ZERO items. MEASURED, not inferred: with the marker-path helper deliberately changed to a single RUN-SHARED path, which is a real cross-run leak, this case still reported PASS and the whole suite still exited 0. A check that passes having examined zero items is precisely the vacuous-pass shape this file exists to close, so the file contained an instance of its own subject. FIX, verified by direct inspection of the working tree: a fourth run identifier now stands in for a DIFFERENT run that is still in flight, its marker is deliberately left un-consumed across the closing run's close path, and the case asserts the foreign marker is genuinely present on disk before proceeding, failing loudly with an explicit harness-assumption-broken message if it is not. The case also uses

the marker path that mark_phase_in_flight prints rather than reconstructing it, so a change to WHERE the marker lives is caught here too.

LVA-146 — Constitutional test — the covenant-propagation test's CLAUDE.md restore did not survive a kill, and its first fix introduced an unbound-variable abort

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/check-constitution/test_covenant_114_propagation.sh **Severity:** P1

WHAT: tests/check-constitution/test_covenant_114_propagation.sh proves its own falsifiability by removing one covenant anchor literal from the REAL root CLAUDE.md, running the real gate, asserting the gate fires, and restoring the file from a backup through a RETURN trap. A RETURN trap fires when the FUNCTION returns; it does not fire when the process is killed. The test was run under a 120-second timeout, exceeded that budget, was terminated mid-probe, and left the covenant anchor for section 11.4.134 replaced by a placeholder string inside the tracked CLAUDE.md. CONSEQUENCE: the next full scripts/check-constitution.sh run reported a covenant-propagation violation, so the test broke the very gate it exercises; the damage was initially MISATTRIBUTED to a stray agent edit before the real cause was found, and a separate documentation-sync stream independently reported the same symptom from the other direction. A test that damages the tree it audits, and whose damage is then blamed on something else, costs more than the coverage it buys. FIX, in two steps, both verified by direct inspection of the working tree. First the trap was widened from RETURN alone to RETURN together with EXIT, INT and TERM. That first attempt introduced a SECOND defect: the handler referenced the function-local backup variable, which is out of scope when an EXIT trap fires late, so the handler aborted with an unbound-variable error under set -u. Corrected by moving the handler to a file-scope restore function at line 78 that reads a file-scope backup variable through a default-value expansion, making it idempotent and safe to run before the backup exists and safe to run twice. RE-VERIFIED at the time of filing: the tracked CLAUDE.md carries the 11.4.134 anchor and zero placeholder occurrences, and scripts/check-constitution.sh runs to completion with exit 0.

LVA-134 — Constitutional gate — the §6.H credential scan flags its own hermetic test fixture, so ci.sh --changed-only exits 1 on this branch

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/check-constitution/test_credential_scan_corpus.sh **Severity:** P2

WHAT: scripts/check-constitution.sh's clause-6.H credential scan (block 6) matches the pattern `private[[:space:]]+object[[:space:]]+[A-Za-z_]*Bridge[[:space:]]*\{` against every tracked file except a fixed exclusion list (.env.example, CHANGELOG.md, .lava-ci-evidence/, the scanner itself, docs/INCIDENT_, and the root and lava-api-go governance docs). tests/check-constitution/test_credential_scan_corpus.sh is TRACKED and is NOT in that exclusion list, and its line 95 contains the fixture literal `printf 'private object CredsBridge {\n}\n' > "$REPO_ROOT/$leak"` — the very string the test plants in order to prove the scanner still catches a real violation. The scanner therefore flags the test that verifies the scanner. EVIDENCE, measured directly: sweeping the pattern across the whole tracked corpus minus the exclusion list yields exactly ONE hit, and it is tests/check-constitution/test_credential_scan_corpus.sh line 95. IMPACT: this is the sole blocking violation making `scripts/ci.sh --changed-only` exit 1 on branch 002-build-test-distribute-pipeline. It is NOT caused by feature 002 — the feature's own 45 files produce zero hits for this pattern. CLOSURE CRITERIA: the fixture is constructed so the literal is assembled at runtime rather than stored verbatim, or the hermetic-fixture path is added to the scanner's exclusion list with a comment naming why, and a test asserts a genuine violation elsewhere is still caught.

LVA-124 — Pipeline 002 — T040 human review gate — amend CLAUDE.md 6.AA + Seventh Law clause 3 for pipeline-run-report path

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** CLAUDE.md **Severity:** P1 **Created-By:** AI

WHAT: feature 002 task T040 is a CONSTITUTIONAL AMENDMENT, not routine code review. It would amend root CLAUDE.md 6.AA (Two-Stage Distribute Mandate) and the Seventh Law clause 3 (Pre-Tag Real-Device Attestation) per research.md R-001, adding an alternate-path clause under which a Pipeline Run Report for the exact commit SHA and versionCode substitutes for human debug-then-release confirmation, scoped

narrowly to distributions this pipeline itself produces. It must include a forensic anchor, a migration plan and a version-bump rationale per the constitution own Amendment procedure. BLAST RADIUS: this is a RELAXATION of an existing anti-bluff principle whose forensic anchor is the 1.2.19-1039 cold-start-crash incident. It also interacts directly with the open firebase-distribute combined-mode finding filed alongside this item: authorizing an unattended combined distribute would authorize exactly the debug-evidence-gates-a-release-APK gap. Review-ready DRAFT text for this amendment is at specs/002-build-test-distribute-pipeline/constitutional-amendments-proposal.md (with .html and .pdf siblings). The draft is NOT applied: CLAUDE.md and .specify/memory/constitution.md are byte-identical to their pre-feature baselines. BLOCKS: feature 002 phase 5 (US3 distribute), specifically task T043. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-125 — Pipeline 002 — T041 human review gate — .specify/memory/constitution.md MAJOR bump 1.1.0 to 2.0.0 matching T040

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .specify/memory/constitution.md **Severity:** P1 **Created-By:** AI

WHAT: feature 002 task T041 updates .specify/memory/constitution.md to match the T040 amendment with a MAJOR version bump 1.1.0 to 2.0.0, per that file own Versioning policy - MAJOR because this is a RELAXATION of an existing principle, not an addition - and updates the Sync Impact Report header accordingly. Paired with T040: neither lands alone. Review-ready DRAFT text for this amendment is at specs/002-build-test-distribute-pipeline/constitutional-amendments-proposal.md (with .html and .pdf siblings). The draft is NOT applied: CLAUDE.md and .specify/memory/constitution.md are byte-identical to their pre-feature baselines. BLOCKS: feature 002 phase 5 (US3 distribute), specifically task T043. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-126 — Pipeline 002 — T048 human review gate — Decoupled Reusable Architecture carve-out for automatic pin advancement

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** CLAUDE.md **Severity:** P1 **Created-By:** AI

WHAT: feature 002 task T048 is a CONSTITUTIONAL AMENDMENT to the Decoupled Reusable Architecture rule in root CLAUDE.md, carving out this pipeline automated submodule-pin-advance step as an explicit, documented exception to "Submodule fetch/pull is an EXPLICIT operator action, never automatic". BLAST RADIUS: 16-plus submodule upstreams with automatic pin advancement. The wave-5 defect filed alongside this item is the concrete demonstration of why this amendment needs careful scoping: the implementing script defaulted to every submodule and would have auto-advanced the constitution governance submodule itself until a default-deny list was added. Review-ready DRAFT text for this amendment is at specs/002-build-test-distribute-pipeline/constitutional-amendments-proposal.md (with .html and .pdf siblings). The draft is NOT applied: CLAUDE.md and .specify/memory/constitution.md are byte-identical to their pre-feature baselines. BLOCKS: feature 002 phase 6 (US4 repository closure), specifically task T055. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-127 — Pipeline 002 — T049 human review gate — .specify/memory/constitution.md update matching T048

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** .specify/memory/constitution.md **Severity:** P1 **Created-By:** AI

WHAT: feature 002 task T049 updates .specify/memory/constitution.md to match the T048 carve-out - the same MAJOR version bump as T041 if landed together, or a follow-up MAJOR if landed separately. Review-ready DRAFT text for this amendment is at specs/002-build-test-distribute-pipeline/constitutional-amendments-proposal.md (with .html and .pdf siblings). The draft is NOT applied: CLAUDE.md and .specify/memory/constitution.md are byte-identical to their pre-feature baselines. BLOCKS: feature 002 phase 6 (US4 repository closure), specifically task T055. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-138 — advance-all-submodules.sh — the governance deny-list covers 1 of 25 submodules while CLAUDE.md requires per-submodule authorization for 16, and 7 are named by no decision at all

Status: Completed (→ Fixed.md) **Type:** Task **Evidence:** tests/pipeline/test_advance_all_submodules.sh **Severity:** P1

WHAT: scripts/advance-all-submodules.sh declares `GOVERNANCE_DENY=("constitution")` at line 256 — a single entry. The repository declares 25 submodules in .gitmodules. CLAUDE.md's Decoupled-Reusable-Architecture pin policy (the pins-frozen paragraph, at line 301 of the current working-tree file) names 16 of them — auth, cache, challenges, concurrency, config, containers, database, discovery, http3, mdns, middleware, observability, ratelimiter, recovery, security, tracker_sdk — as pins-frozen by default whose bumps require deliberate per-submodule operator authorization. The same paragraph carries the standing Q9 waiver for helixqa (always-track-upstream) and routes constitution through the CONST-049 pipeline. EVIDENCE, computed directly from .gitmodules against that paragraph: 25 submodules total; 16 require per-submodule authorization; 18 are named by SOME operator decision; and 7 are named by NO operator decision at all — doc_processor, llm_orchestrator, llm_provider, llms_verifier, panoptic, superspec, vision_engine. IMPACT: the script's mechanical policy and the governance document's written policy do not agree about 16 submodules, and neither of them says anything about the remaining 7, so an advance run's blast radius is not bounded by anything an operator has actually authorized. CLOSURE CRITERIA: the deny-list (or an equivalent per-submodule policy table) enumerates every one of the 25 with an explicit stance, the 7 unnamed submodules gain a recorded operator decision, and a test asserts the script's policy table and the governance document agree entry for entry.

LVA-135 — Pre-push hook — SIGPIPE under pipefail turns a MATCH into a NO-MATCH, silently skipping four constitutional gates on large commits

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pre-push/test_large_commit_gate_sigpipe.sh **Severity:** P0

WHAT: .githooks/pre-push declares `set -euo pipefail` (line 34) and then evaluates four gate conditions in the shape `git diff-tree --no-commit-id --name-only -r "$sha" | grep -q….` `grep -q` exits at its first match; the still-writing producer is then killed by SIGPIPE and reports 141; pipefail promotes 141 to the pipeline status; the enclosing `if` therefore evaluates FALSE on exactly the commits the gate exists to reject. EVIDENCE, measured directly on real commit f1a2c3627e443dd4d2c16da59e7fe3a3bd71b9d7 (1530 paths / 97783 bytes of --name-only output): the if-condition status is 141 with PIPESTATUS=(141 0), while the identical pipeline without pipefail returns 0; across 20 trials the gate saw the match 0 times and missed it 20 times. A threshold scan puts the flip at the 65536-byte pipe buffer: at or above it the producer cannot finish writing before grep exits, so the gate is blind on every trial. AFFECTED SITES AND GATES: line 63 (Check 1, Local-Only CI/CD hosted-CI config introduction), line 135 and line 137 (Check 6, §6.Y bump-first ordering), line 167 (Check 7, §6.Z distribute test-evidence companion), line 210 (Check 10, §6.AK cycle-coverage — the `|| continue` form, where the SIGPIPE status makes the gate skip the commit entirely). WHY EXISTING TESTS MISS IT: the hermetic fixtures under tests/pre-push/ build small synthetic commits whose path listing stays far below the pipe buffer, so the SIGPIPE path is never entered. Full record: .lava-ci-evidence/bluff-hunt/2026-08-23-cycle-gate-shaping-scripts.json finding F1.

LVA-120 — Pipeline 002 — firebase-distribute.sh --debug-and-release ships the RELEASE APK on DEBUG-channel evidence

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/firebase/test_combined_mode_retired.sh **Severity:** P0 **Created-By:** AI

WHAT (verified verbatim in the real script this session, line by line, not inferred): scripts/firebase-distribute.sh line 56 - "--debug-and-release|--both) MODE="both"; shift". Line 204 - the constitution 6.AA staging gate is guarded on [["$MODE" == "release" && -f "$LAST_VERSION_DEBUG_FILE"]], so in combined mode it NEVER EVALUATES. Lines 322-323 - "case "$MODE" in release) AK_CHANNEL="release" ;; *) AK_CHANNEL="debug" ;; esac", so the constitution 6.AK cycle-coverage gate (which also subsumes the 6.Z evidence presence, commit-SHA match and 24h freshness checks) is pointed at the DEBUG channel. Lines 435, 444 and 513 - the RELEASE APK is version-asserted, resolved and uploaded whenever MODE is release OR both. NET EFFECT: a combined distribute ships the R8-minified RELEASE APK while the only device-evidence gate that runs checks DEBUG-channel evidence. That is mechanically the same setup as the 1.2.19-1039 incident (the constitution 6.Z

forensic anchor), in which a release APK crashed on every cold launch because release-variant behaviour was never gated. PRE-EXISTING: this is a property of scripts/firebase-distribute.sh and is NOT introduced by feature 002. REPRODUCTION: tests/firebase/repro_mode_both_channel_gap.sh. It NEVER executes firebase-distribute.sh (that would attempt a real Firebase upload); it greps the four real decision lines verbatim, evaluates that extracted logic per MODE, and exits 2 if those lines ever stop existing rather than silently testing a stale copy. Its exit semantics are inverted on purpose and documented in its header: 0 means the gap was REPRODUCED (expected today), 1 means it is GONE and the harness should be converted into a proper regression test, 2 means extraction failed and it can say nothing truthful - exit 2 is never read as "fixed". REPRODUCED OUTPUT: mode debug - release APK sent no, 6.AK channel debug, 6.AA staging gate no; mode release - release APK sent yes, 6.AK channel release, 6.AA staging gate yes; mode both - release APK sent YES, 6.AK channel DEBUG, 6.AA staging gate NO. The release row is the control that proves this is not a false alarm: --release-only DOES evaluate both gates on the correct channel. The gap is specific to the combined mode - precisely the mode an unattended pipeline would use. IMPACT ON FEATURE 002: task T043 was specified to invoke exactly --debug-and-release and would have inherited the hole; any amendment authorizing an unattended combined distribute would authorize exactly this. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.

LVA-136 — Constitutional gate — the propagation check has no floor on its submodule corpus, so its verdict depends on checkout state

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/check-constitution/test_propagation_target_floor.sh **Severity:** P2

WHAT: scripts/check-constitution.sh builds `propagation_targets` (lines 241-254) from 5 FIXED entries (CLAUDE.md, AGENTS.md, and the three lava-api-go governance docs) and then extends it by looping over `submodules/*/CLAUDE.md` guarded by `[[ -f "$sub" ]]`. When the submodules are uninitialised that guard drops every submodule doc, and the §6.N / §6.O / §6.R / §6.S / §6.X propagation loops that consume the array PASS on a corpus of 5 — while the identical propagation drift FAILS when the submodules ARE initialised. The gate's verdict is a function of checkout state rather than of the tree's compliance. EVIDENCE, measured directly: 22 submodule CLAUDE.md files are present in this checkout against 5 hardcoded targets, so an uninitialised tree silently drops 22 of 27 targets (81 percent) with nothing to detect it. THE OMISSION

IS LOCAL, NOT A PROJECT-WIDE CONVENTION: the same script already carries the zero-floor pattern for a sibling gate at lines 149-152, refusing a §6.H credential scan that examined zero tracked files, and scripts/verify-all-constitution-rules.sh carries the same floor for the sweep's gate count at lines 287-290. The propagation block is the one that lacks it. Full record: .lava-ci-evidence/bluff-hunt/2026-08-23-cycle-gate-shaping-scripts.json finding F3.

LVA-130 — Pipeline 002 — an interrupted run finalizes as outcome PASS on a truncated phase prefix

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** tests/pipeline/test_interrupted_run_never_passes.sh **Severity:** P0

WHAT: finalize_run_report() in scripts/pipeline/lib/run-report.sh decides the run outcome with `(.phases | length) > 0 and (.phases | all(.result == "PASS"))` (jq branch) and the identical `len(phases) > 0 and all(p.get("result") == "PASS" ...)` in the python3 fallback branch. That predicate closes the EMPTY-phases case but NOT the TRUNCATED-PREFIX case: a run killed after an early phase has a non-empty phases array in which every recorded entry is PASS, so it satisfies the predicate and finalizes PASS. EVIDENCE, measured directly: .lava-ci-evidence/pipeline-runs/2026-08-23T10-15-30Z/report.json carries `outcome: "PASS"` with exactly ONE phase (precondition/PASS), an empty build_artifacts array, and an all-zero evidence_summary (total 0, passed 0, failed 0, skipped 0, rejected_by_anti_bluff 0). A run that built nothing and tested nothing reports the same outcome token as a complete green run, which is the §6.J shape: nothing was learned, reported as nothing failed. STATUS: under active fix by the conductor in this session; tests/pipeline/test_interrupted_run_never_passes.sh is present in the working tree as the accompanying regression coverage. CLOSURE CRITERIA: outcome resolves to FAIL (or a distinct INTERRUPTED token) whenever the recorded phase sequence is a proper prefix of the phase set the invocation requested, with the regression test asserting it against a real truncated report rather than a synthetic one.

LVA-147 — Section 6.AA clause 8 — conditions (A) and (H) deadlock: the run-report schema forbids exactly the field (H) mandates

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** CLAUDE.md **Severity:** P2

RESOLVED 2026-08-26 by rewording, NOT by amending the schema (operator decision). The deadlock was real: clause 8 condition (A) required the run report to validate against pipeline-run-report.schema.json, which sets additionalProperties false and defines no residual-gap property, while condition (H) mandated exactly that field — so no run could qualify. Measured at the time: the generator emitted the field 0 times and 0 of 48 on-disk reports carried it, so (H) failed for the simpler reason and the net effect failed SAFE. RESOLUTION: the §6.AA amendment of 2026-08-26 withdrew former condition (D) and re-lettered (E)-(I) to (D)-(H); the residual-gap condition is now (G) and records the gap through build_artifacts[].build_output_path — a property the schema already defines AND lists as required (pipeline-run-report.schema.json, build_artifacts.items.properties.build_output_path). No residual-gap field is emitted and no property outside the schema is written, so the report continues to validate under condition (A). The alternative candidate distributions[].evidence_ref was rejected on evidence: distributions[] is empty at gate-evaluation time because the gate runs before any upload, so a condition anchored there would be unevaluable at the only moment clause 8 is evaluated. VERIFIED against the applied tree: phase-05-distribute.sh CONDITION_IDS=(A B C D E F G H) with no (I); tests/pipeline/test_phase_05_distribute_gate.sh and test_phase_05_distribute_gate_vacuity.sh both ALL CHECKS PASSED exit 0 (41 assertions).

LVA-150 — check-workable-items.sh — the CM-WORKABLE-ITEMS-SYNC gate diffs only Issues.md and Fixed.md, so both Summary documents can be stale while the gate reports OK

Status: Fixed (→ Fixed.md) **Type:** Bug **Evidence:** scripts/check-workable-items.sh **Severity:** P1 **Created-By:** AI

WHAT: scripts/check-workable-items.sh -- the CM-WORKABLE-ITEMS-SYNC gate -- verifies DB-to-Markdown sync by invoking the canonical binary's diff subcommand with exactly two document arguments, --issues "$ISSUES" and --fixed "$FIXED". The four generated tracker surfaces are Issues.md, Fixed.md, Issues_Summary.md and Fixed_Summary.md. The gate therefore covers two of four. The two Summary documents -- the section 11.4.12 open-only Type-by-Status tally and the section 11.4.53 closed-only tally -- are regenerated by the binary's export subcommand and are NEVER compared by the gate. CONSEQUENCE: after any DB mutation performed without a subsequent export, Issues_Summary.md and Fixed_Summary.md retain the PREVIOUS generation's item text and the PREVIOUS generation's tallies, while the gate reports exit 0 and prints its OK line naming the DB as validated and in sync. The gate's PASS is truthful about the two documents it examines and silent about the two it does not, and a reader of the PASS line has no way to tell those apart. DISCOVERED BY INSPECTION during the LVA-008 correction earlier this session: the item's status and description had been changed in the DB, Issues.md reflected the change, and Issues_Summary.md continued to present LVA-008 as Operator-blocked carrying the superseded description -- with the gate green throughout. IMPACT: the Summary documents are the surfaces a human opens for a tally, so a stale Summary is precisely the surface a human reads for a number and does not re-derive. A gate that passes while an artifact it is understood to guard is stale is the section 6.J class: the green signal reports safety that was never measured. NOTE ON THE OPERATOR-FACING EXPORT PATH: the binary's export defaults its output directory to the REPOSITORY ROOT, and this repository tracks BOTH a root-level and a docs/ copy of all four tracker documents, so a regeneration that targets only one location leaves the other pair stale in the same silent way. CLOSURE CRITERIA: the gate compares all four generated surfaces (or regenerates them into a temporary directory and asserts byte-equality against the tracked copies at BOTH tracked locations), with a hermetic fixture that mutates the DB, regenerates only Issues.md and Fixed.md, and asserts the gate FAILS -- which it does not do today.